

Optimization of Quantitative Trading Strategies through 3D Visualization Techniques

By

Chan Li Yang



**FACULTY OF COMPUTING AND
INFORMATION TECHNOLOGY**

**TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY
KUALA LUMPUR**

**ACADEMIC YEAR
2023/2024**

Optimization of Quantitative Trading Strategies Through 3D Visualization Techniques

By

Chan Li Yang

Supervisor:
Miss Fatin Izzati Ramli

A project report submitted to the
Faculty of Computing and Information Technology
in partial fulfillment of the requirement for the
Bachelor of Computer Science (Hons.) in Data Science

Department of Mathematical and Data Science
Faculty of Computing and Information Technology
Tunku Abdul Rahman University of Management and Technology
Kuala Lumpur

Copyright by Tunku Abdul Rahman University of Management and Technology.

All rights reserved. No part of this project documentation may be reproduced, stored in retrieval system, or transmitted in any form or by any means without prior permission of Tunku Abdul Rahman University of Management and Technology.

Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources had been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.



Chan Li Yang

ID: 22WMR09009

Abstract

This project seeks to innovate in the field of quantitative trading by constructing a 3D visualization tool designed to tackle the intricate challenge of parameter optimization in trading strategies. By employing state-of-the-art data science and sophisticated visualization methods, the tool will enable users to explore and adjust the complex parameter space that controls algorithmic trading systems. The primary goal is to enable the exploration of optimal parameter setups using an engaging and interactive three-dimensional model. In order to facilitate improved analytical skills and more strategic decision-making in the field of quantitative trading, this model seeks to crystallize complex data correlations into a visual form that highlights the most effective combinations. This project will culminate in a critical tool that will enable trading strategy deployment to perform at a higher level by converting numerical data that is abstract into concrete visual insights.

Acknowledgment

I would like to sincerely thank everyone who has supported me in finishing this project and show my thanks. First and foremost, I would like to express my gratitude to Miss Fatin Izzati Ramli and Mr. Choong Yun Loong, who served as my project supervisor and moderator, for their important advice, encouragement, and support during the project. Their knowledge, perceptions, and kind criticism have been invaluable in helping me to refine my work. I would like to express my sincere gratitude to my family and friends for supporting me during my academic career. My inspiration and strength have always come from their undying love, support, and encouragement. Finally, I would want to express my gratitude to each and every participant for so kindly lending me their time and sharing their experiences and wisdom. Their contributions were essential to the success of this project. I want to thank everyone again for their assistance in getting this project finished.

Table of Contents

1.0 Introduction	2
1.1. Background	2
1.2 Problem Statement	3
1.3 Project Objectives	4
1.4 Scope of Project	5
1.5 Methodology Overview	6
1.6 Chapter Summary	7
2.0 Literature Review	9
2.1 Foundations of Quantitative Trading	9
2.2 Advanced Analytical Techniques in Trading	9
2.3 Expanding the Repertoire of Trading Strategies	10
2.4 Machine Learning in Financial Parameter Optimization	10
2.5 Gaussian Process Models for Parameter Selection	10
2.6 Overfitting and Cross-Validation	11
2.7 Chapter Summary and Evaluation	11
3.0 Methodology and Requirements Analysis	13
3.1 Research Methodology	13
3.1.1 Data Collection	13
3.1.2 Data Preprocessing	14
3.1.3 Strategies Selection and Implementation	15
3.1.4 Parameter Optimization Using Bayesian Optimization Techniques	22
3.1.5 Metrics for Evaluation	23
3.1.6 Visualization of Parameter Optimization Results	24
3.2 Requirements Gathering	25
3.2.1 Literature Insights	25
3.3 Requirements Analysis	26
3.3.1 Functional Requirements	26
3.3.2 Non-functional Requirements	27
3.4 Analytical Techniques	28
3.5 Chapter Summary and Evaluation	31
4.0 System Design	33
4.1 Bayesian Optimization Architecture	33
4.1.1 Model Selection	33
4.1.2 Acquisition Function	34
4.1.3 Visualization	35
4.2 Grid Search CV Architecture	36
4.2.1 Parameter Grid Setup	37
4.2.2 Cross-Validation Mechanism	38
4.2.3 Performance Metric	39
4.2.4 Optimal Parameter Selection	39

4.2.5 Visualization	40
4.3 Flow Chart	41
Data Collection	41
Data Preprocessing	41
Strategy Development	42
Parameter Optimization	42
Evaluation and Visualization	42
Real-World Testing	43
4.4 User Interface	43
4.4.1 User Interface Components	44
1. File Upload Component	44
2. Strategy Selection Bar	44
3. Timeframe Selection	44
4. Optimization Method Selection	45
5. Run Optimization Button	45
4.4.2 Interactive Features	47
4.4.3 Utility	48
4.5 Chapter Summary and Evaluation	48
5.0 Implementation and Result	50
5.1 Implementation	50
5.1.1 Data Extraction	50
5.1.2 Data Preprocessing	51
5.1.3 Strategy Deployment	52
5.1.4 Parameter Optimizaiton	53
5.1.5 3D Visualization	56
5.1.6 Performance Metrics	58
5.2 Result	61
5.2.1 Grid Search CV Best Parameter Combinations	61
5.2.2 Bayesian Optimization Best Parameter Combinations	61
5.2.3 Evaluation of Grid Search CV	62
5.2.4 Evaluation of Bayesian Optimization	62
5.3 Chapter Summary and Evaluation	63
6.0 Deployment and Real World Testing	65
6.1 Deployment (Streamlit)	65
6.2 Real World Testing	67
6.2.1 Preparation for Testing	67
6.2.2 Strategy Testing	68
6.2.3 Creating an Instance on Cybotrade Cloud	69
6.2.3 Real-World Testing Results	71
6.3 Chapter Summary and Evaluation	74
7 Discussions and Conclusion	77
7.1 Summary	77
7.2 Achievements	78

7.3 Contributions	79
7.4 Limitations and Future Improvements	80
7.5 Issues and Solutions	81
References	82
Appendix	85

Chapter 1

Introduction

1.0 Introduction

The advent of algorithmic trading strategies has heralded a new era in financial markets, characterized by a reliance on mathematical models that facilitate automated decision-making. These sophisticated strategies are integral to modern trading but demand meticulous optimization of a vast array of parameters, from entry and exit thresholds to risk-reward ratios and beyond. Each parameter's calibration is critical to a strategy's success, with even slight modifications potentially yielding significant impacts on outcomes. The intricate and high-dimensional nature of the parameter space presents a significant challenge, rendering the identification of optimal settings a complex, often insurmountable task without the aid of advanced tools.

1.1. Background

The digitization of financial markets has given rise to algorithmic trading, a field where precision and speed are paramount. In this landscape, quantitative trading strategies stand out for their ability to dissect and respond to market signals with a level of accuracy that human traders cannot match. At the core of these strategies lie myriad parameters that dictate their operation. The calibration of parameters such as entry and exit thresholds, risk-reward ratios, and rolling window periods is not merely a routine task; it's an intricate exercise that can significantly impact the profitability of trades (Chan, 2013). However, the complexity and dimensionality of the parameter space create a formidable challenge, one that demands more than traditional optimization techniques can offer.

As quantitative strategies grow increasingly complex, so does the necessity for advanced methods to navigate and optimize their parameters. Traditional methods often fall short, shackled by their static nature and lack of dynamic visual feedback (López de Prado, 2018). Such methods typically involve a retrospective analysis, like backtesting against historical data, which carries the inherent risk of overfitting — a statistical mirage that can render a model effective in hindsight but ineffective in forecasting future market behavior. It's this limitation that the proposed project seeks to overcome, through the development of a 3D visualization tool that not only enhances the exploratory experience but also brings to light the subtle

interdependencies between parameters that two-dimensional representations fail to capture.

1.2 Problem Statement

Current optimization techniques in quantitative trading are largely dependent on two-dimensional data representation or tabular forms, which can obscure the intricate relationships between multiple parameters. This limitation often results in a cumbersome and time-consuming process that can be prone to errors, potentially leading to suboptimal trading performance and increased operational costs.

Quantitative trading strategies, which are essentially sophisticated mathematical models, hinge upon the precise calibration of a myriad of parameters to dictate buy and sell decisions in the frenetic world of financial markets. The ideal choice of parameters, such as rolling window periods, risk-reward ratios, volatility estimations, entry and exit thresholds, and so on, is crucial to the effectiveness of these algorithms. Determining the optimal parameters for a trading strategy in order to optimize earnings is the process of optimization. (Strategic Optimization of Parameters in Quantitative Trading | AlinaKhay, n.d.) Both a benefit and a drawback of this granularity of parameters is that, because of the intricate linkages and sheer size of the multidimensional parameter space, they add a great deal of complexity even while they provide fine-grained control over the trading logic. Financial markets are notoriously unpredictable and tumultuous, which makes it easy for static parameter settings to become ineffective over time. This adds to the complexity of the situation.

In this high-stakes setting, traditional parameter optimization techniques frequently fail. In order to assess performance, these techniques usually entail backtesting strategies against historical market data. However, there is always a chance of overfitting, and this procedure can be computationally taxing. Overfitting arises when a model is overly customized to historical data, resulting in erroneous patterns that don't translate to new market circumstances and poor out-of-sample performance. (Chan, 2014) Furthermore, traditional methods typically yield results in formats that are less suited for quick thinking, like spreadsheets or static graphs, which are unable to adequately represent the multidimensional nature of the data.

The tendency of these approaches to settle on locally optimal parameter values that are suboptimal globally but optimal within a narrow section of the parameter space is another significant constraint. These kinds of solutions are brittle and could break down in the face of the inherent volatility and unpredictability of financial markets. Strategies may therefore not only perform poorly but also expose traders to unanticipated risks.

Thus, the search for a stable and profitable trading strategy necessitates a novel approach to parameter optimization—one that goes beyond the confines of conventional techniques by combining dynamic visualization and deft parameter space exploration. Both intuitive and flexible enough to guide users through intricate data landscapes and adjust to changing market trends, this strategy must be. It should ensure that the tactics developed are sustainable and profitable by optimizing not just for past profitability but also for stability and resilience against market fluctuations.

1.3 Project Objectives

In the current era of financial markets dominated by algorithmic trading, the demand for sophisticated tools to optimize trading strategies is paramount. This project aims to develop a state-of-the-art 3D visualization tool to transform the traditionally static and complex process of trading strategy parameter adjustment into a dynamic and intuitive experience. The project objectives are stated below:

1. **Visual Representation of Parameter Space:** The tool will transform complex data arrays into a clear, three-dimensional map of the parameter space, providing a comprehensive view that highlights the interactions and impacts of different parameters. This visualization aids in quickly identifying optimal trading conditions and discerning subtle patterns not visible in two-dimensional formats.
2. **Optimization of Strategy Analysis:** Utilizing advanced computational techniques, the tool will facilitate real-time adjustments and visual feedback on parameter changes, simplifying the tuning process. This allows for rapid

strategy iteration and refinement, streamlining the transition from hypothesis to actionable insights.

3. **Enhancement of Decision-Making Processes:** By clarifying complex datasets' interdependencies, the tool will enhance users' ability to make informed decisions quickly. It serves as a decision support system that not only optimizes but also encourages the innovation of trading strategies based on real-time data analysis.

These objectives collectively aim to advance the field of quantitative trading by making strategy optimization more accessible and actionable through innovative visualization technology.

1.4 Scope of Project

The scope of this project is carefully defined to ensure a focused approach towards enhancing quantitative trading strategies through technological innovation. Central to this endeavor is the development of a user-centric 3D visualization platform that is meticulously tailored to meet the needs of modern quantitative trading. This platform is not just a visual aid; it is an analytical tool that will cater to the nuanced demands of trading strategy development and optimization.

The project will achieve its vision through two primary avenues:

1. Integration of Machine Learning for Parameter Optimization

At the heart of the platform lies the integration of sophisticated machine-learning algorithms. These algorithms are designed to sift through vast amounts of historical market data to detect patterns and derive insights that can inform optimal parameter configurations. The goal is to harness the predictive power of machine learning to offer a data-driven foundation upon which traders can build and refine their strategies. These algorithms will account for the probabilistic nature of financial markets, ensuring that the tool not only identifies the parameters that historically have yielded success but also considers the uncertainty and variability inherent in market behavior.

2. Interactive User Interface for Strategy Exploration

The second component of the project's scope is the development of an intuitive interface. This interface will be the conduit through which users interact with the complex world of quantitative trading strategies. It is designed to empower users to select, adjust, and visualize a wide array of strategy parameters and instantly observe the potential outcomes of their interdependencies. The interface will facilitate a hands-on exploration of strategies, enabling users to test and hypothesize in an environment that closely mimics the dynamics of real-world markets. Through this interactive platform, traders can visualize the impact of their choices, experiment with 'what-if' scenarios, and gain confidence in the strategies they choose to deploy.

In its entirety, the scope of this project is to create a tool that stands at the intersection of innovation and practicality. The 3D visualization platform will not only encapsulate the rigorous technical requirements of quantitative trading but also embody the user-centric design principles that make complex optimization tasks more approachable and actionable for traders and analysts alike.

1.5 Methodology Overview

This project's methodology is based on an extensive framework that guarantees a solid strategy for optimizing quantitative trading methods. Important components of the approach consist of:

1. Comprehensive research assessment

To identify gaps and current issues in the field of quantitative trading strategy optimisation, the project begins with a thorough assessment of the body of existing research. This review aids in comprehending the shortcomings of current methods and directs the creation of novel remedies to these shortfalls.

2. Data Science and Visualisation Techniques

By utilizing cutting-edge data science techniques, the project fine-tunes the parameters of several trading strategies using statistical approaches like Bayesian Optimisation. Using state-of-the-art visualization tools more especially, creating a 3D visualization model to complements this. This model plays a crucial role in offering comprehensible insights into the intricate dynamics of strategy optimization, enabling a visual evaluation of the effects of various factors on trading strategy performance.

3. Empirical Validation Using Historical Data

To make sure that the optimisation tools created are both theoretically and practically sound, the approaches are put to the test by using historical market data. To evaluate the efficacy and flexibility of trading methods, a variety of market circumstances are simulated during testing. The tool's accuracy and efficacy in improving strategy optimisation are thoroughly assessed thanks to the use of actual historical data, offering a solid foundation for practical implementation.

These elements serve as the foundation of the project's methodology, guaranteeing a thorough approach to the creation and improvement of data-driven, graphically understandable trading methods. This comprehensive approach not only makes the solutions more technically sound, but it also guarantees that consumers in the trading industry can easily access and utilise them.

1.6 Chapter Summary

This introductory chapter lays the foundation for the project by stressing the critical role of parameter optimization in quantitative trading and acknowledging the shortcomings of existing methods. It sets forth the project's objectives and scope and sketches an overview of the methodologies to be employed. The subsequent chapters will delve deeper into the literature review, describe the methodology employed in detail, elaborate on system design and implementation, and document the testing procedures and final evaluations, leading to a discussion on the deployment of the project and its findings

Chapter 2

Literature Review

2.0 Literature Review

The development of quantitative trading has fundamentally changed the way financial markets operate, requiring ever more advanced tools for strategy optimization. The information presented in this chapter is a summary of four seminal papers that have improved our knowledge of algorithmic trading, strategy optimization, and the use of cutting-edge analytical technology.

2.1 Foundations of Quantitative Trading

Quantitative trading reduces emotional trading strategy decision-making by combining intricate mathematical models with historical data analysis. These techniques' fundamental characteristic is their reliance on thorough backtesting to compare theoretical models to actual market behavior(Ernestp. chan, 2021). This method emphasizes how important it is to manage data accurately and choose the right software tools, two essential components that support the healthy growth of algorithmic trading operations. These fundamental ideas paved the way for the creation of complex instruments, such as the suggested 3D visualization platform, which improves the parameter optimization procedure.

2.2 Advanced Analytical Techniques in Trading

Traditional trading tactics have changed as a result of the incorporation of machine learning, optimization, and sophisticated statistical tools into trading strategies. These methods are essential for tackling issues with market impact, transaction costs, and trade success rates in general. Thorough use of Transaction Cost Analysis (TCA) is an example of the careful analysis needed to improve deals' cost-effectiveness(Kissell, 2020). By using these sophisticated methodologies, tools that simplify difficult optimization problems through user-friendly visual interfaces are developed, which has a direct impact on the efficacy and efficiency of trading strategies.

2.3 Expanding the Repertoire of Trading Strategies

A wide range of quantitative trading tactics show the flexibility and diversity needed to be successful in a variety of asset classes and market environments. Navigating a wide range of tactics with an adjustable instrument is essential, from simple momentum techniques to sophisticated arbitrage (Zura Kakushadze, 2021). This variation is taken into account in the design of the suggested 3D visualization tool, which allows traders to investigate and refine various tactics in a dynamic, interactive setting.

2.4 Machine Learning in Financial Parameter Optimization

Specialized machine-learning approaches are required to efficiently extract relevant insights from financial information due to their distinct properties. By encapsulating the core of intricate datasets, Gaussian model-based trading parameter optimization demonstrates the potential of these methods to improve the selection process. (López de Prado, 2018) In order to build robust trading strategies, these models provide a probabilistic modeling and prediction approach to market fluctuations.

2.5 Gaussian Process Models for Parameter Selection

A reliable foundation for parameter optimisation in the creation of trading strategies is offered by Gaussian process models. The dynamic and intricate character of financial markets is best suited for them because of their adaptability in managing a variety of data distributions and incorporating prediction uncertainty. (López de Prado, 2018) Customizing strategies that can endure the volatility of market conditions requires the ability to adjust hyper-parameters in these models.

2.6 Overfitting and Cross-Validation

A typical issue known as "overfitting" occurs when a model fits a limited number of data points too closely and is unable to generalize to additional data. In order to guarantee the accuracy of financial models, overfitting must be addressed. In situations where data may not be uniformly and independently distributed, methods such as cross-validation are crucial for confirming the robustness of models.(López de Prado, 2018) These approaches are essential to preserving the integrity of parameter optimization procedures because they guarantee that techniques are both practically and conceptually sound.

2.7 Chapter Summary and Evaluation

To summarise, this overview of the literature has explained how theoretical developments and real-world applications in Gaussian models and machine learning help traders optimize their quantitative trading systems' parameters. With the integration of these advanced analytical methods, the suggested 3D visualization tool seeks to transform parameter selection and improve quantitative trading decision-making abilities.

Chapter 3

Methodology and Requirements Analysis

3.0 Methodology and Requirements Analysis

This chapter will cover the requirements analysis and techniques used in the creation of quantitative trading strategies. This covers an in-depth talk on feature engineering, data collection, and preprocessing methods, and choosing and executing different trading strategies. We will also investigate strategy parameter optimization and the assessment measures that are employed to gauge their effectiveness. In order to guarantee the system's dependability, usefulness, and scalability in actual market circumstances, the chapter also describes the functional and non-functional requirements.

3.1 Research Methodology

Research Methodology describes the thorough research process that was employed to create and assess Bitcoin quantitative trading methods. From data collection and preprocessing to trading strategy implementation and optimization, it covers the whole process. The chapter stresses that in order to guarantee the efficacy and flexibility of the trading system, it is critical to employ solid data, a variety of trading methods, and sophisticated optimization techniques. In order to make sure the designed system is dependable and effective in actual market conditions, it also emphasizes the evaluation criteria for the strategies.

3.1.1 Data Collection

For this project, historical price data of Bitcoin over the past four years is utilized, specifically captured on 1 Hour timeframe. The data is sourced from CCXT (CryptoCurrency eXchange Trading Library) using the “ccxt” library, which provides a reliable and programmatically accessible method to fetch historical market data. This approach ensures both accuracy and reliability in the data used for analysis.

Data Period and Significance:

1. Start Date

The dataset begins on May 11, 2020, coinciding with Bitcoin's third halving event. This event is a significant milestone in Bitcoin's lifecycle, as it reduces the reward for mining new blocks by half, thereby decreasing the rate at which

new bitcoins are generated. This halving process can potentially influence market sentiment and price, making it a critical starting point for analyzing market behaviors post-halving.

2. Four-Year Span

Covering four years, the dataset encompasses different market phases, including both bull and bear markets. This duration is essential to capture a wide array of market dynamics. For example, a bull market is the period where the market experiences sustained upward trajectories and provides insights into how Bitcoin behaves in favorable conditions. Analyzing such periods helps in understanding the factors that drive market enthusiasm and price increases. In a Bear Market, conversely, periods of prolonged downward trends are crucial for testing the resilience of trading strategies under adverse conditions. It allows for the evaluation of risk management strategies and the robustness of indicators during market downturns.

Incorporating data from both bull and bear markets is vital for developing a well-rounded trading strategy. This approach ensures that the strategy is not overly optimized for a single market condition but is robust enough to adapt to varying market dynamics. By taking into account diverse market conditions, the strategy minimizes risk and enhances reliability, making it applicable across different market cycles.

3.1.2 Data Preprocessing

Cleaning

The raw dataset may contain missing values due to various issues like data transmission errors or anomalies in data collection. These missing values are handled using the following methods:

Forward-Fill Method

For most missing entries, the forward-fill method is employed. This technique propagates the last observed non-null value forward until another non-null value is

encountered. This approach is particularly useful in time-series data where it is reasonable to assume that changes between consecutive data points are incremental.

Removal of Initial Missing Values

The values are eliminated from the dataset if they are missing values at the beginning where there are no prior data points available for a forward-fill. In cases where a fair estimate cannot be made, this ensures that the analysis is based on complete and correct data without introducing bias from imputed data.

Timestamp Standardization

Coordinated Universal Time has been applied to all timestamps in the dataset (UTC). This standardization takes care of any discrepancies that could result from the data sources' various time zones. It guarantees that all data points are in sync with each other in terms of time, which is essential for precise analysis, particularly when combining data from several sources or contrasting worldwide events.

Feature Engineering

We define a new feature called 'Moving Averages'. Moving averages smooth out price data by creating a constantly updated average price. This is particularly useful to identify trends. Both simple moving averages (SMA) and exponential moving averages (EMA) might be calculated to provide insights into short-term and long-term price movements.

3.1.3 Strategies Selection and Implementation

There are major two types of strategy categories that are applied in quantitative trading which are the momentum strategy and mean reversion strategy. Momentum or trending strategies aim to capitalize on market trends. They assume that assets which have been moving in a particular direction will continue to move in that direction. Mean Reversion (Ranging) Strategies are based on the assumption that prices and returns eventually move back towards the mean or average. These strategies work well in non-trending markets where prices fluctuate within a range. So, in this project,

it will involve 3 strategies under momentum strategies and 2 strategies under mean reversion strategies.

1. MACD (Moving Average Convergence/Divergence) Strategy

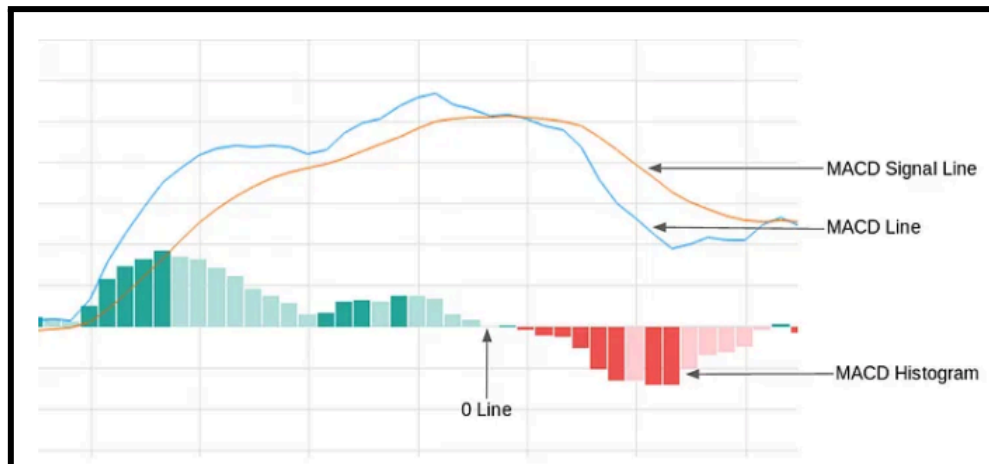


Figure 3.1.1 MACD Strategies Chart

Introduction

The MACD is a trend-following momentum indicator that shows the relationship between two moving averages of an asset's price. The MACD is calculated by subtracting the 26-period Exponential Moving Average (EMA) from the 12-period EMA. The result of this calculation is the MACD line. A signal line, which is the 9-period EMA of the MACD, is then plotted on top of the MACD line, which can function as a trigger for buy and sell signals. (Best MACD Trading Strategy (Moving Average Convergence Divergence) | by Quantified Strategies | Medium, Sep 23, 2023.)

Buy Signal

Typically occurs when the MACD crosses above its signal line, indicating a potential upward movement in price.

Sell Signal

This occurs when the MACD crosses below its signal line, suggesting a potential downward trend.

2. Moving Average 2 Line Cross Strategy



Figure 3.1.2 Moving Average 2 Line Cross Strategy Chart

Introduction

The Moving Average 2 Line Cross Strategy is a technical analysis strategy that depends on the idea that moving averages crossing each other signals changes in the trend direction of the asset's price (Maximizing Profitability of Moving Average Crossover Trading Strategy: What Moving Averages to Use? | by Facundo Joel Allia Fernandez | Medium, Apr 13, 2023.). It uses two moving averages of varying durations to find possible buy and sell signals. The essential elements comprise a short-term moving average, computed over a shorter timeframe like 10 or 20 days, which responds swiftly to price fluctuations, and a long-term moving average, computed over a longer timeframe like 50 or 200 days, which is less susceptible to recent price fluctuations and offers a more refined trend line that represents the general direction of the market.

Buy Signal

Generated when the short-term moving average crosses above the long-term moving average, indicating a potential rise in the asset's price.

Sell Signal

Generated when the short-term moving average crosses below the long-term moving average, indicating a potential fall in the asset's price.

3. ADX DI Crossover Strategy



Figure 3.1.3 ADX DI Crossover Concept

Introduction

A technical trading approach called the ADX and DI Crossover Strategy uses the Directional Indicators (+DI and -DI) and the Average Directional Index (ADX) to find possible trading opportunities. The +DI and -DI evaluate the trend's direction, whereas the ADX gauges a trend's strength. By creating buy and sell signals based on the crossover of these indicators, this technique seeks to seize trend-following chances. To increase the strategy's efficacy in capturing significant trends, the ADX threshold is employed to filter out signals during weak or non-trending periods.

Buy Signal

1. There is a significant trend when the ADX number is higher than the predetermined threshold.
2. The +DI (Directional Indicator Positive) is greater than the -DI (Directional Indicator Negative), suggesting an upward trend.

3. The previous value of +DI was less than or equal to the previous value of -DI, indicating a potential crossover.

Sell Signal

1. **There is a significant trend when the ADX number is higher than the predetermined threshold.**
2. The -DI (Directional Indicator Negative) is greater than the +DI (Directional Indicator Positive), suggesting a downward trend.
3. The previous value of -DI was less than or equal to the previous value of +DI, indicating a potential crossover.

4. Mean Reversion Strategy

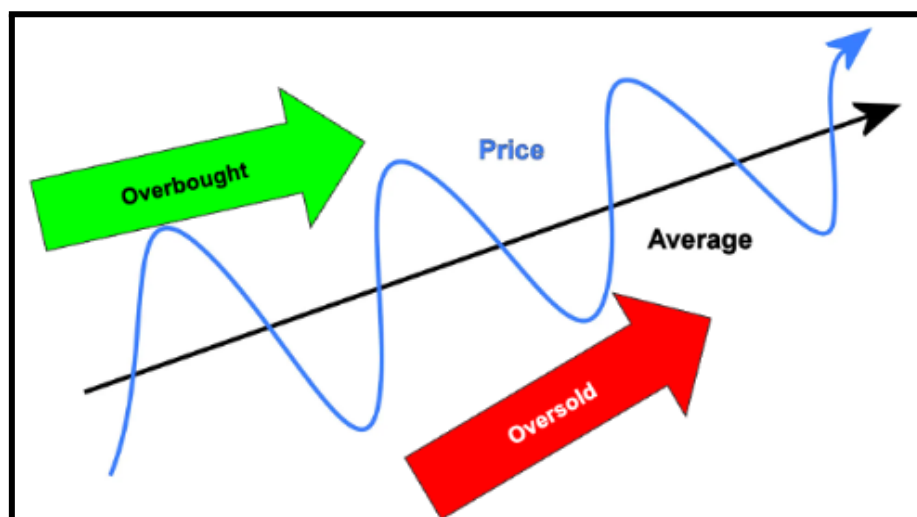


Figure 3.1.4 Mean Reversion Concept

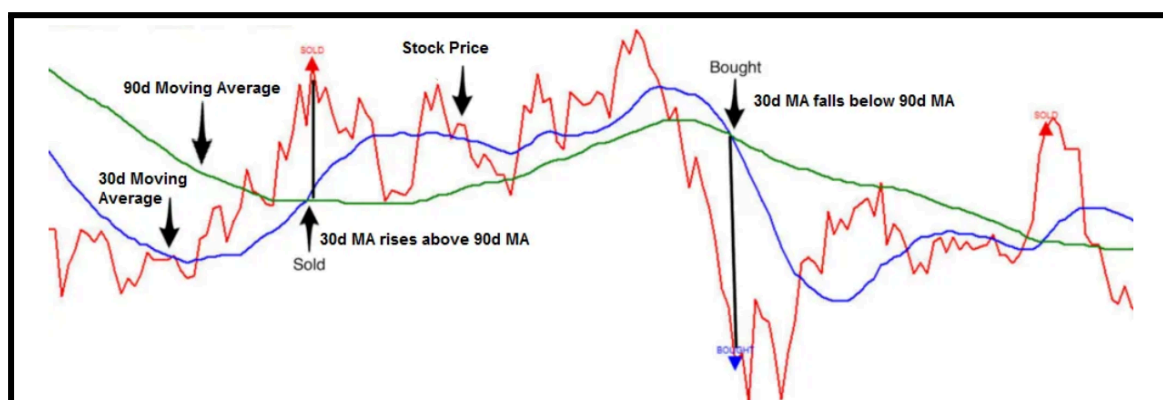


Figure 3.1.5 Mean Reversion Strategy Chart

Introduction

The mean reversion strategy relies on the idea that prices fluctuate around a mean or average level and that transitory deviations from this mean will eventually cause the price of a financial asset to return to its historical average (Mean Reversion: Simple Trading Strategies Part 1 | by Auquan | Auquan | Medium, Aug 30, 2017.). The technique looks for significant deviations from the historical average and treats them as anomalies that will eventually correct themselves. The historical average, which is the average price of the asset over a given historical period and is typically chosen to smooth out short-term fluctuations and provide a reliable mean, is one of the main components. The other is the deviation from the mean.

Buy Signal

When the price is unusually low compared to its historical average, suggesting it will rise back to the mean.

Sell Signal

When the price is unusually high, suggesting it will drop back to the mean.

5. Bollinger Band + RSI Strategy

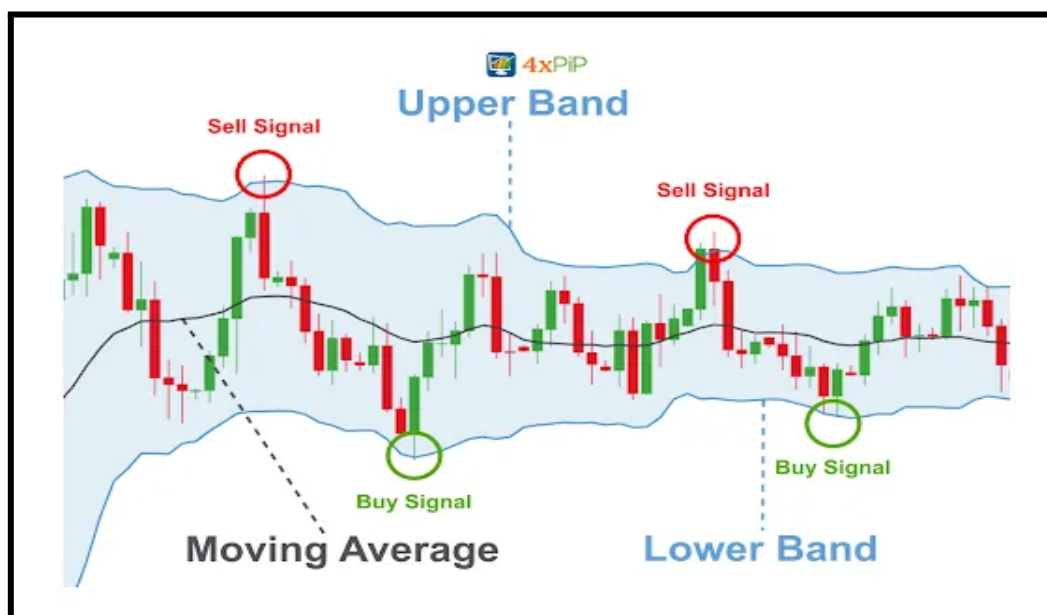


Figure 3.1.6 Bollinger Band Strategy Concept



Figure 3.1.7 RSI Strategy Concept

Introduction

This strategy combines Bollinger Bands with the Relative Strength Index (RSI) to identify overbought or oversold conditions in a range-bound market. Bollinger Bands consists of a middle band being a simple moving average, surrounded by two standard deviation bands. These bands expand and contract based on market volatility(Quant Trading with Python Using Bollinger Bands | by Judy T Raj | Medium, Feb 27, 2024.).RSI helps identify whether the asset is overbought (typically above 70) or oversold (typically below 70)(Unleashing the Power of RSI: An In-Depth Manual for Trading with Relative Strength Index | by Unschooled Trader | Medium, May 30, 2023.).

Buy Signal

When the price touches or breaks through the lower Bollinger Band and the RSI is below 30.

Sell Signal

When the price touches or breaks through the upper Bolling Band and the RSI is above 70.

6. Zscore SMA Strategy

Introduction

The Z-Score SMA Strategy measures the price's deviation from its Simple Moving Average (SMA) using the Z-score. The SMA establishes a baseline for the average price of the asset over a given time period. By quantifying the current price's divergence from the average by the number of standard deviations, the Z-score enables traders to spot profitable trades based on notable variances. With the assumption that significant price deviations from the historical average can indicate future price reversals or continuations, this approach seeks to profit from such deviations.

Buy Signal

The Z-score of the current price is greater than the specified Z-score threshold for example 0.75, indicating that the current price is significantly above the SMA.

Sell Signal

The Z-score of the current price is less than the negative of the specified Z-score threshold for example -0.75, indicating that the current price is significantly below the SMA.

3.1.4 Parameter Optimization Using Bayesian Optimization Techniques

For the optimization of objective functions that are costly to evaluate, one approach is Bayesian optimization. This characteristic makes it perfect for modifying trading strategy hyperparameters. It is particularly well suited for optimization problems with unknown derivatives and those that are noisy and expensive to analyze. When deciding where to sample next, Bayesian Optimisation makes informed decisions by using a probabilistic model to anticipate the objective function.

In this research, threshold levels for RSI and MACD, window sizes for moving averages, and other trading strategy parameters are adjusted using Bayesian

optimization. The procedure entails establishing a plausible range for every parameter, followed by iterative testing of parameter values to identify the combination that maximizes trading success as shown by metrics like the cumulative return or the Sharpe ratio.

The method is based on creating a probabilistic model, usually a Gaussian Process (GP), which maps parameter values to a loss probability and utilizes this information to choose parameter values that will be most effective to evaluate in the real system. The GP becomes more accurate at predicting which areas of parameter space are worthwhile to explore as the number of data increases. Compared to grid or random search, this method dramatically lowers the number of evaluations required, conserving resources and possibly resulting in the discovery of superior parameter settings.

3.1.5 Metrics for Evaluation

In assessing the effectiveness of trading strategies, it is essential to measure not only the returns but also the risks involved. The primary metrics used for this purpose include:

Profitability (Average Return)

This metric computes the total profits in order to assess the strategy's absolute financial performance. It offers a clear indicator of how much money the strategy has gained, making it possible to evaluate financial success directly.

Risk-Adjusted Returns (Sharpe Ratio)

A popular statistic for evaluating an investment's return in relation to risk is the Sharpe ratio. It is computed by deducting the risk-free rate (such as the yield on a three-month Treasury bill) from the investment's return and dividing the result by the standard deviation of returns for the investment. Since the investor is getting more return per unit of risk, a greater Sharpe ratio denotes a more desirable risk-adjusted return.

Drawdown

Usually expressed as a percentage, this metric calculates the biggest single decline in the investment portfolio's value from peak to bottom. Understand a strategy's potential risk by looking at its drawdowns, which show the maximum amount an investor could lose in the worst-case situation. This aids in evaluating the volatility of the strategy as well as the level of financial and emotional resilience needed while asset values are down.

3.1.6 Visualization of Parameter Optimization Results

In order to assess the efficacy of various parameter sets and make well-informed judgments regarding strategy modifications, it is imperative to visualize the outcomes of parameter optimization. In order to provide light on the parameter space and trading strategy performance, visualization tools are being created for this project. These tools will allow the results of the Bayesian Optimisation and Grid Search CV process to be graphically displayed.

The visualization includes:

1. **Performance Landscape:** A 3-dimensional figure illustrating the relationship between parameter values and performance measurements, such as the Sharpe ratio or total returns. This facilitates the discovery of patterns and trends in the parameter effects on performance.
2. **Convergence Plot:** Plotting the optimization process over a number of rounds demonstrates how close the algorithm is to determining the ideal set of parameters. It is very helpful for confirming that the Bayesian Optimisation and Grid Search CV procedure works as intended.
3. **Parameter Correlation Heatmaps:** Heatmaps are a useful tool for visualizing the relationship between several parameters and how those parameters affect the overall performance of a strategy. This can reveal parameter interactions that individual analysis is unable to reveal.

By offering insightful input that can be utilized to improve the optimization process, these visualizations help to ensure that the final trading strategy is both highly

optimized and robust for the target market circumstances, in addition to assisting with the instant study of optimization results.

3.2 Requirements Gathering

To guarantee that the trading methods and the corresponding visualization tool are both efficient and user-friendly, the project's requirements were acquired using a thorough and multifaceted methodology. The project's specifications were developed with considerable input from the literature already in existence, industry best practices, and the state of financial analytics technology.

3.2.1 Literature Insights

The project's underlying ideas and technical methods were greatly influenced by the literature review. Scholarly works, industry studies, and reputable texts pertaining to financial engineering and quantitative trading have furnished invaluable perspectives about the present condition and constraints of extant trading tactics and instruments. Research, for example, has shown that it is crucial to combine momentum and mean-reversion techniques in order to profit from various market conditions. This has a direct bearing on the choice to incorporate a variety of trading strategies in the project. The project's 3D visualization component was added as a result of the literature on advanced data visualization emphasizing the necessity of dynamic and interactive tools in the financial markets. Important papers on the developments in financial machine learning, such as those by Lopez de Prado, provided insights into how to apply statistical techniques and machine learning algorithms for strategy optimization and risk management.

Moreover, the literature review also revealed that overfitting is a common problem in quantitative models, which influenced the project's focus on strong backtesting frameworks and the application of out-of-sample data testing to guarantee the models' performance in practical situations. In order to make sure that these criteria were

included into the project's evaluation phase, it also included instructions on the critical metrics for assessing trading strategies, such as net profitability, Sharpe ratio, and maximum drawdown.

After a thorough examination of existing scholarly and applied materials, the project's specifications were precisely established in order to fill in the gaps in the available trading strategy tools and take advantage of the most recent developments in financial theory and technology. The project's solid academic foundation and high relevance and applicability to contemporary financial markets have been guaranteed by this meticulous requirements-gathering approach.

3.3 Requirements Analysis

In order to ensure the success and effectiveness of the project, a thorough analysis of both functional and non-functional requirements was conducted. This analysis helps define the specific needs that the project must meet, guiding the development process and ensuring that the final product aligns with the expected standards and goals.

3.3.1 Functional Requirements

The functional requirements of the project are directly related to the capabilities that the system must have to perform its intended functions effectively. Here is the listed of functional requirements of the project:

1. **Data Processing:** The system must be capable of ingesting and processing historical price data of Bitcoin and potentially other cryptocurrencies. This involves reading data from various sources, handling different formats, and ensuring data integrity during the process.
2. **Indicator Calculation:** Essential to the trading strategies, the system must calculate various technical indicators such as MACD, RSI, and moving averages accurately and efficiently. These calculations are crucial for the subsequent steps in the trading strategy.

3. **Signal Generation:** Based on the calculated indicators and predefined criteria, the system must generate trading signals. These signals indicate potential buy or sell opportunities, guiding the trading decisions.
4. **Trade Simulation:** The system should have the capability to simulate trading activities based on the generated signals. This includes executing trades within a simulated environment to assess the strategy's performance without the financial risk of real trading.
5. **Visualization Tools:** To aid in analysis and decision-making, the system must provide visualization tools that can display both raw data and trading indicators in an understandable format. This requirement is vital for the interactive exploration of data and strategy performance.

3.3.2 Non-functional Requirements

Non-functional requirements are those that outline the behavior or performance expectations of a system without being specifically tied to a feature or function of the system. The non-functional requirements focus on the quality attributes of the system are listed below:

1. **Performance:** The system must process data and perform calculations within a timeframe that is acceptable for trading. This includes rapid processing of incoming data and quick generation of trading signals to capitalize on market opportunities.
2. **Usability:** The interface for interacting with the system, including setting parameters and viewing results, must be intuitive and user-friendly. This ensures that users can effectively utilize the system without extensive training.
3. **Scalability:** Given the vast amounts of data involved in trading, the system must be scalable. It should handle increases in data volume without significant degradation in performance. This includes the ability to scale up for additional cryptocurrencies if the project expands.

4. **Reliability:** The system must be reliable, with minimal downtime and accurate performance, as errors can lead to significant financial loss.
5. **Security:** Data security must be a priority, particularly when dealing with financial information. The system must ensure data integrity and protect against unauthorized access.

These functional and non-functional requirements work together to guarantee that the project will produce a reliable and efficient instrument for quantitative trading. Through addressing these particular requirements, the project hopes to deliver a system that may be modified and enhanced in the future to accommodate shifting market conditions and ambitions.

3.4 Analytical Techniques

The project's evaluation phase uses a number of advanced analytical methodologies and metrics to analyze and comprehend the trading strategies' performance throughout live and backtested testing. It is vital to employ these tactics to guarantee that the solutions exhibit theoretical soundness and function effectively in actual market situations.

Descriptive and inferential statistics are used in the statistical analysis of the trading strategy to assess the strategy's profitability and risk. The statistical significance of the data is ascertained by performing hypothesis tests and computing the means, variances, and correlations of the returns. Understanding the underlying return distribution and potential risk variables that could impact the strategy's performance is made easier with the use of statistical analysis.

To measure the performance of the trading strategies comprehensively, several key metrics are used:

1. Cumulative Returns

This metric calculates the total return gained or lost over the period of the strategy's operation, providing a straightforward measure of overall performance(Cumulative Return: Definition, Calculation, and Example, October 10, 2020.). It is calculated as:

$$\frac{(Current\ Price\ of\ Security) - (Original\ Price\ of\ Security)}{Original\ Price\ of\ Security}$$

Figure 3.4.1 Cumulative Return Formula

2. Maximum Drawdown

This measure evaluates the worst single decline in the value of the portfolio from peak to trough before the achievement of a new peak. It is essential for calculating the likelihood of large losses. The formula for maximum drawdown is:

$$MDD = \frac{Trough\ Value - Peak\ Value}{Peak\ Value}$$

Figure 3.4.2 Maximum Drawdown Formula

where the peak value is the highest value reached before the largest drop, and the trough value is the lowest value reached before a new peak is achieved(Maximum Drawdown (MDD) Defined, With Formula for Calculation, April 07, 2024.).

3. Sharpe ratio

The Sharpe Ratio is used to understand the return of the strategy adjusted for its risk, providing a measure of risk-adjusted performance relative to a risk-free asset(Sharpe Ratio - How to Calculate Risk-Adjusted Return, Formula, n.d.). The formula is:

$$\text{Sharpe Ratio} = \frac{(R_x - R_f)}{\text{StdDev } R_x}$$

Figure 3.4.3 Sharpe Ratio Formula

Where:

R_x = Expected portfolio return,

R_f = Risk-free rate of return

StdDev R_x = Standard deviation of portfolio return/volatility

4. Calmar ratio

The Calmar Ratio is a performance metric used to evaluate the risk-adjusted return of an investment or trading strategy (Wallstreetmojo team, 2024). It sheds light on how well an investment has done in relation to its risk, paying particular attention to losses. The formula is:

$$\text{Calmar Ratio} = \frac{\text{Average Annual Rate of Return}}{\text{Maximum Drawdown}}$$

Figure 3.4.3 Calmar Ratio Formula

Where:

Average Annual Rate of Return = The compounded annual growth rate of the investment or strategy.

Maximum Drawdown = The maximum observed loss from a peak to a trough before a new peak is reached.

3.5 Chapter Summary and Evaluation

This chapter describes the methodology employed to collect and preprocess historical Bitcoin data starting on May 11, 2020, to maintain data integrity utilizing timestamp standardization, forward-fill techniques, and cleaning. In order to create momentum and mean reversion trading strategies, it presents feature engineering using indicators such as historical volatility and moving averages. The chapter describes how to choose and use methods like Bollinger Band + RSI, Mean Reversion, Flash, Moving Average 2 Line Cross, MACD, and their corresponding buy and sell signals. While evaluation indicators like profitability, Sharpe ratio, and drawdown offer a thorough analysis of strategy effectiveness, Bayesian optimization techniques are employed for parameter tuning to maximize strategy performance.

Drawing from literature and industry best practices, the requirements gathering section makes sure the project satisfies functional as well as non-functional requirements. It focuses on performance, usability, scalability, reliability, and security, as well as data processing, indicator calculation, signal generation, trade simulation, and visualization tools. To thoroughly assess the trading methods, analytical tools such as statistical analysis and key performance metrics are used. By combining theoretical ideas with real-world applications, this organized process guarantees the creation of a reliable and efficient quantitative trading system.

Chapter 4

System Design

4.0 System Design

Chapter 4 provides a comprehensive overview of the Bayesian Optimization, Grid Search CV architecture and the project workflow. This chapter will outline the flow of data through the system, detail the functionality of each module, and demonstrate how these modules interact to form a cohesive and functional trading system.

4.1 Bayesian Optimization Architecture

Bayesian Optimization is a sophisticated statistical technique used in the parameter optimization of trading strategies within this project. It is perfect for fine-tuning trading algorithms when each evaluation may involve a lot of computing, such as backtesting over huge datasets because it is specifically made to optimize complex objective functions that are costly to evaluate.

4.1.1 Model Selection

Gaussian Processes are used in Bayesian Optimisation to model the unknown objective function. A group of random variables with a joint Gaussian distribution that is limited in number is called a Gaussian process. Its covariance function and mean function define it.

Mean Function $m(x)$

This function shows the mean or predicted value of the function over the domain that you are trying to optimize. Assuming this to be 0 in many actual implementations is standard practice when there is no prior knowledge about the function's behavior.

Covariance Function $k(x, x')$

It defines the output covariance of the Gaussian Process at the input points x and x' and is also referred to as the kernel. The selection of the kernel is important because it encodes certain assumptions about the function, like periodicity and smoothness. The Squared Exponential, Matérn, and Rational Quadratic kernels are examples of common kernels(Seeger, 2004).

$$k(\mathbf{x}_p, \mathbf{x}_q) = \sigma_f^2 \exp \left(-\frac{1}{2} (\mathbf{x}_p - \mathbf{x}_q)^\top M (\mathbf{x}_p - \mathbf{x}_q) \right) + \sigma_n^2 \delta_{pq}, \quad (5.1)$$

where $\boldsymbol{\theta} = (\{M\}, \sigma_f^2, \sigma_n^2)^\top$ is a vector containing all the hyperparameters,² and $\{M\}$ denotes the parameters in the symmetric matrix M . Possible choices for the matrix M include

$$M_1 = \ell^{-2} I, \quad M_2 = \text{diag}(\boldsymbol{\ell})^{-2}, \quad M_3 = \Lambda \Lambda^\top + \text{diag}(\boldsymbol{\ell})^{-2}, \quad (5.2)$$

where $\boldsymbol{\ell}$ is a vector of positive values, and Λ is a $D \times k$ matrix, $k < D$.

Figure 4.1.1 Squared Exponential Kernel Formula

4.1.2 Acquisition Function

By balancing the exploration of new regions with the exploitation of known regions, the acquisition function determines the next best place to sample. It comes from the posterior distribution that the GP offers. The often selected options are the Expected Improvement (EI) and Probability of Improvement (PI).

Expected Improvement (EI)

The concept of Expected Improvement (EI) identifies points that could potentially surpass the best-observed value. It takes into account the uncertainty of the surrogate model's mean forecast as well as the predicted improvement over the present best result (Step-by-Step Guide to Bayesian Optimization: A Python-Based Approach | by Okan Yenigün | Medium, Jun 7, 2023.). At any given position in the input space, this function calculates the expected percentage improvement over the current best observation.

$$EI(x, Q) = E_Q[\max(0, \mu_Q(x_{\text{best}}) - f(x))].$$

Figure 4.1.2 Expected Improvement (EI) Formula

Where:

- x_{best} as the location of the lowest posterior mean.
- $\mu_Q(x_{\text{best}})$ as the lowest value of the posterior mean.

Probability of Improvement (PI)

The possibility that a point will surpass the current best value is estimated by the Probability of Improvement (PI). Taking into account the uncertainty in the surrogate model, it evaluates the difference between the mean prediction and the current best value. (Step-by-Step Guide to Bayesian Optimization: A Python-Based Approach | by Okan Yenigün | Medium, Jun 7, 2023.) In contrast to Expected Improvement, which expresses the expected difference over the present best achievable, PI only calculates the probability that any given point would exceed the best-observed result as of right now. The formula is calculated as :

$$PI = \Phi(\nu_Q(x)),$$

where

$$\nu_Q(x) = \frac{\mu_Q(x_{best}) - m - \mu_Q(x)}{\sigma_Q(x)}.$$

Figure 4.1.3 Expected Improvement (EI) Formula

Where:

- X_{best} as the location of the lowest posterior mean.
- $\mu_Q(X_{best})$ as the lowest value of the posterior mean.
- Φ as the cumulative distribution function (CDF) of the standard normal distribution.

4.1.3 Visualization

Deep insights into the processes of optimisation can be gained by visualizing the GP model's developing landscape and the acquisition function across a series of iterations. The model's uncertainty and potential location of the global optimum, for example, can be discerned via 3D surface plots of the GP's mean and variance across the parameter space, which can help inform strategic choices for the next location of sampling.

4.2 Grid Search CV Architecture

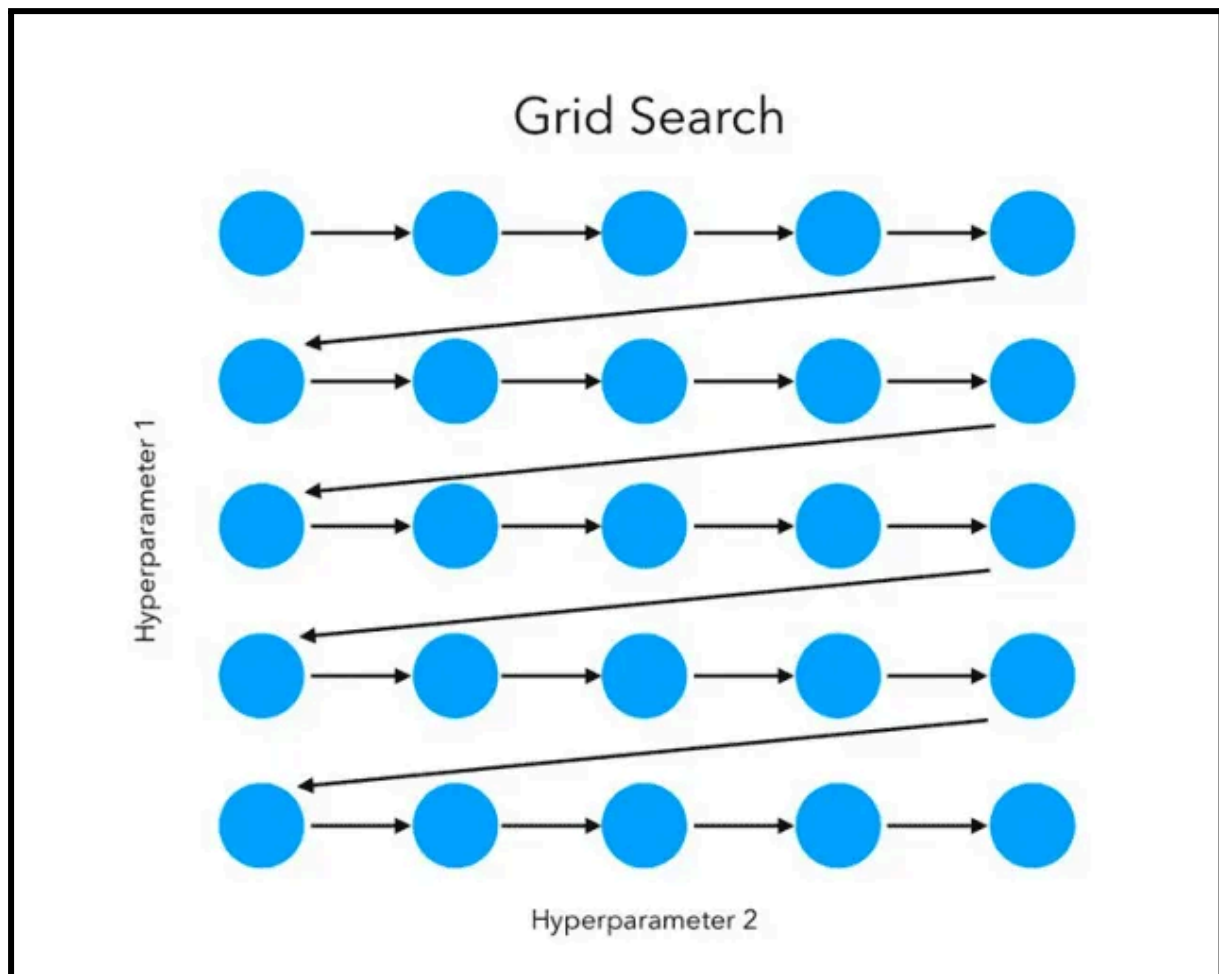


Figure 4.2.1 Process of Grid Search

Grid Search CV is an all-inclusive parameter optimization technique used to methodically assess each and every combination of values within a given grid of parameters (Nabin Adhikari, 2022). For models where the performance could be greatly impacted by every parameter configuration, this thorough approach is crucial. In contrast to probabilistic techniques such as Bayesian Optimization, Grid Search guarantees that no possible combination of parameters within the grid is missed. This methodical approach is a valuable asset to the optimization toolkit since it works well in situations when parameter influence is significant.

4.2.1 Parameter Grid Setup

The fundamental framework for methodical investigation and assessment of various parameter combinations of a model is provided by the parameter grid configuration in Grid Search CV. This configuration entails constructing an extensive grid that delineates the potential values for every parameter of concern, so generating a multi-dimensional space in which every point signifies a distinct collection of parameters. When optimizing a trading strategy, for instance, the grid may have parameters like moving average length, buy/sell thresholds, and take-profit or stop-loss levels. For example, moving averages can range from 10 to 100 days in steps of 10, or stop-loss levels can range from 0.5% to 5% in increments of 0.5%. Each parameter can have a range of values. This methodical technique guarantees that no possible combination is missed, offering a comprehensive analysis of the parameter space.

The significance of this configuration cannot be emphasized because it directly affects the Grid Search's capacity to determine the ideal parameters. Through a methodical assessment of all potential combinations, the grid search makes sure that the final model is optimized for optimal performance as measured by the chosen performance metric—in this example, the Sharpe Ratio. In order to make sure that the performance evaluation is reliable and isn't unduly fitted to a specific subset of data, every possible combination of parameters is used to train and validate the model using cross-validation. This procedure is essential for creating trading strategies that work in practice as well as theory in real-world situations. The efficacy of the strategy can be greatly impacted by the identification of possibly unexpected interactions between parameters made possible by the thorough investigation of the parameter space.

4.2.2 Cross-Validation Mechanism



Figure 4.2.2 Cross-Validation Mechanism

A crucial component of the Grid Search CV is the cross-validation procedure, which guarantees the robustness and dependability of performance assessments of various parameter combinations. This process, which usually uses K-fold cross-validation, entails splitting the data set into 'K' distinct folds, or subsets. The remaining fold is used as a test set to assess the model's performance once it has been trained on 'K-1' of these folds (Rahul, 2024). Each of the 'K' folds is utilized as the test set exactly once during the 'K' repetitions of this operation. Through the process of iterating over many combinations of training and validation sets, cross-validation serves to reduce the likelihood that the performance of the model will be unduly influenced by any one subset of the data. This makes sure that a more thorough depiction of the model's capabilities throughout the entire dataset serves as the foundation for the performance measures.

Moreover, Grid Search CV's cross-validation architecture offers a strong means of evaluating a trading strategy's applicability to fresh, untested data. This is particularly important in financial applications where it is imperative that a strategy be highly adaptable due to the unpredictable nature of market conditions. Every cycle of training and validation provides information about how the strategy functions in marginally different scenarios, simulating the unpredictability that one may anticipate in actual trading situations. This rigorous validation procedure helps identify the most stable and efficient parameter configurations in addition to boosting trust in the model's performance indicators, which in this case are usually centered

on maximizing the Sharpe Ratio. The average of all folds' performance measures offers a fair picture that prioritizes parameter combinations that yield the best returns while maintaining consistency and dependability across a range of market conditions.

4.2.3 Performance Metric

In Grid Search CV for this project, the effectiveness of each parameter combination is quantitatively evaluated using the Sharpe Ratio. This metric is particularly crucial as it measures the risk-adjusted return of the trading strategies, providing a more comprehensive assessment of performance than mere returns. The Sharpe Ratio is calculated as the average return earned in excess of the risk-free rate per unit of volatility or total risk. Utilizing this metric allows for the comparison of strategies on a risk-adjusted basis, ensuring that the strategy not only yields high returns but does so with an acceptable level of risk.

The optimal parameter set is chosen based on the combination that achieves the highest Sharpe Ratio across all cross-validation folds. This selection process is instrumental in identifying parameters that enhance the financial performance of the trading strategy while managing volatility effectively. By focusing on maximizing the Sharpe Ratio, Grid Search CV aligns the model optimization process with the ultimate goal of achieving superior risk-adjusted returns, which is vital in the volatile domain of trading strategies.

This approach underscores the importance of not just achieving profitability but doing so in a manner that judiciously balances the returns with the risks involved, a fundamental aspect often emphasized in quantitative trading. By meticulously assessing the impact of parameter variations on the Sharpe Ratio, the project ensures that the trading strategies not only perform well under typical market conditions but also maintain robustness during market fluctuations.

4.2.4 Optimal Parameter Selection

The process of selecting the best parameters for Grid Search CV involves extensive testing with different parameter combinations and cross-validation techniques. The combination of parameters that consistently produces the highest Sharpe Ratio throughout all cross-validation folds is chosen as the best after each potential combination has been assessed and their performances measured, usually utilizing the Sharpe Ratio in this project.

This stage is crucial because it guarantees that the parameters selected are not only the best ones conceptually but also the ones that have worked best in real-world scenarios with a range of situations, as demonstrated by the cross-validation procedure. By placing a strong emphasis on maximizing the Sharpe Ratio, the selection process naturally gives top priority to the trading strategy's stability and risk-adjusted performance in addition to profitability. This guarantees that the resulting model is dependable and strong, capable of effectively identifying and seizing market opportunities.

4.2.5 Visualization

In the Grid Search CV process, visualization is essential since it offers clear and practical insights into the outcomes of parameter tuning. This stage effectively illustrates how different parameter combinations affect the model's performance, especially with regard to the Sharpe Ratio, by using both heatmaps and 3D visualization tools. The 3D visualizations are particularly beneficial for displaying the multidimensional character of the parameter space, allowing stakeholders to visually investigate the relationships between various parameters and their effects on the trading strategy's outcomes.

Heatmaps are also an effective tool for providing a color-coded matrix summary of the grid search results, with each cell representing a combination of parameters and the colors denoting performance level. This facilitates swift identification of the grid's best and worst performing regions. By clearly showing the ideal parameter settings, these visualizations not only support the final decision-making process but also enable a deeper comprehension of the strategy's robustness and parameter sensitivity. These observations are crucial for improving the model even further and can direct future changes to the trading strategy, making sure it stays flexible and successful in a range of market circumstances.

4.3 Flow Chart



Figure 4.2 Project Flow Chart

Data Collection

In this first phase, historical Bitcoin price data is gathered from reliable sources such as CCXT (CryptoCurrency eXchange Trading Library). Over the last four years, the data includes periods of notable market events such as Bitcoin halvings, as well as everyday price swings.

Data Preprocessing

The gathered data is cleaned, standardized, and made ready for analysis at this phase. To guarantee data quality and consistency, methods like filling in missing values, aligning timestamps to UTC, and converting prices into logarithmic returns are used.

Strategy Development

At this point, multiple trading strategies are created in response to distinct market movements. There are two types of techniques: mean-reversion strategies that use indicators like Bollinger Bands and RSI, and momentum-based strategies that use moving averages and MACD.

Parameter Optimization

Each trading strategy's parameters are adjusted using Bayesian Optimisation or Grid Search CV to maximize performance. Based on performance indicators from prior evaluations, Bayesian Optimisation and Grid Search CV selects values that most likely result in the best outcomes by using a probabilistic model to efficiently explore the parameter space.

Evaluation and Visualization

Following parameter optimization, the techniques' effectiveness is assessed by backtesting, which makes use of historical data to gauge performance. Calculations are made to determine important performance indicators such as maximum drawdown, Sharpe ratio, and cumulative returns. These evaluations' outcomes, in particular the effects of various parameter choices, are displayed in 3D graphs such as the one seen above. This graphic aids in comprehending the performance landscape and the multidimensional parameter space by showing how various parameters affect the strategy's results.

Real-World Testing

Testing the optimized trading methods in an actual trading environment(Cybotrade Cloud) is the last step. This phase is essential for verifying the strategies' effectiveness and resilience in real-time market scenarios, making sure they function well in both historical simulations and real trading.

4.4 User Interface

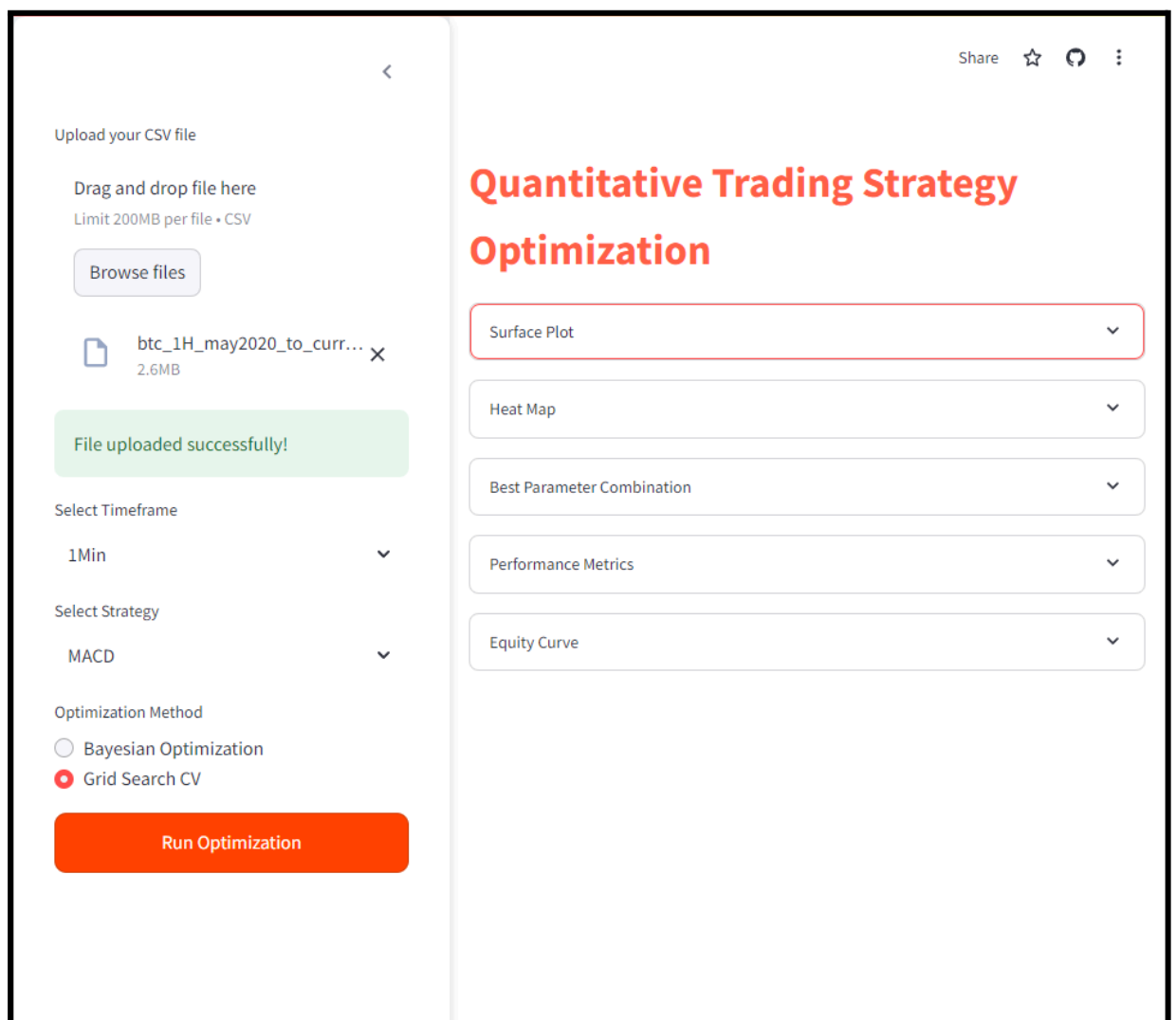


Figure 4.3 Draft of User Interface Design

4.4.1 User Interface Components

1. File Upload Component

➤ **Functionality**

The file upload component allows users to upload a CSV file containing the historical data required for optimization. The system processes the uploaded data to evaluate the strategy's performance.

➤ **Purpose**

Providing a custom dataset enhances flexibility, allowing users to perform optimizations on specific datasets of interest, tailored to their trading scenarios.

2. Strategy Selection Bar

➤ **Functionality**

The Strategy Selection Bar is a dropdown menu labeled "Select Strategy." It allows users to select from a list of available trading strategies. The available strategies are MACD, Moving Average Cross, ADX DI Crossover, Mean Reversion, Bollinger Band + RSI, and Z-Score SMA. Each strategy has different parameters that can be optimized.

➤ **Purpose**

This component allows users to switch between different strategies for comparison. Users can observe how each strategy performs under various parameter configurations, making it easier to determine which strategy is most effective under certain conditions.

3. Timeframe Selection

➤ **Functionality**

The Timeframe Selection dropdown allows users to choose the timeframe for the data used in the optimization. Timeframes include 1H, 4H, 1Day, and more, with each corresponding to different data granularity.

➤ **Purpose**

By selecting different timeframes, users can analyze the strategy performance across various market conditions, catering to short-term and long-term trading strategies.

4. Optimization Method Selection

➤ **Functionality**

This component provides two radio buttons labeled "Bayesian Optimization" and "Permutation Testing (Grid Search CV)." Users can select one of these optimization techniques to find the best parameters for the selected strategy. Reversion, Bollinger Band + RSI, and Z-Score SMA. Each strategy has different parameters that can be optimized.

➤ **Purpose**

The availability of multiple optimization methods allows users to apply different techniques to find the optimal parameters for their trading strategy, either through the efficient exploration provided by Bayesian Optimization or the exhaustive search of Permutation Testing.

5. Run Optimization Button

➤ **Functionality**

The "Run Optimization" button triggers the optimization process based on the selected strategy, timeframe, and optimization method.

➤ **Purpose**

This button initiates the parameter search process, allowing users to discover the best parameters for the selected strategy.

6. 3D Graph

➤ **Functionality**

The central feature of the user interface is the 3D graph, which visually represents the parameter optimization landscape. The graph plots the Sharpe Ratio on one axis and the two key parameters on the other axes.

➤ **Purpose**

The 3D graph visualization aids users in understanding the relationship between parameter values and strategy performance. It allows for easy identification of parameter combinations that yield the best Sharpe Ratio, making it easier to interpret complex relationships.

7. Best Parameter Combination

➤ Functionality

This section, presented within an expander, displays the best-performing parameter values in a tabular format after the optimization process is complete.

➤ Purpose

This component provides a quick summary of the optimal parameters, allowing users to easily reference the most effective parameter configuration identified during the optimization process.

8. Performance Metrics

➤ Functionality

The Performance Metrics section presents a table within an expander that details key metrics such as Sharpe Ratio, Average Return, Maximum Drawdown, Calmar Ratio, and Number of Trades.

➤ Purpose

The performance metrics provide users with a comprehensive overview of how the strategy performs with the optimized parameters. This information is crucial for evaluating the effectiveness and robustness of the trading strategy.

9. Surface Plot

➤ Functionality

Depending on the chosen optimization method, this section visualizes the optimization results. It can show a 3D scatter plot for Bayesian Optimization or a surface plot for Grid Search CV.

➤ **Purpose**

This visualization allows users to see the entire optimization landscape, helping them understand how different parameter combinations impact the Sharpe Ratio. The plot also highlights the optimal point to indicate where the best performance was achieved.

10. Heatmap (for Grid Search CV only)

➤ **Functionality**

For users who select the Grid Search CV method, a heatmap is displayed within an expander to show the Sharpe Ratio across different combinations of parameters.

➤ **Purpose**

The heatmap provides a visual representation of how parameter combinations influence the strategy's performance, offering a more detailed and intuitive understanding of the optimization results.

4.4.2 Interactive Features

The user interface is meant to be interactive, enabling users to actively adjust and investigate many factors rather than merely viewing static data:

1. **Dynamic Updates**

The 3D graph and related metrics (Best Combination and Sharpe Ratio) instantly update to reflect the outcomes of the new strategy's optimization when a user chooses a different approach from the dropdown menu.

2. **Zoom and Rotate**

By interacting with the 3D graph, users can observe the optimization landscape from various perspectives and identify local maxima or minima that might not be apparent from a single viewpoint.

4.4.3 Utility

With the help of this user interface, users will be able to visually investigate and assess the efficacy of various trading methods by optimizing their parameters. In order to facilitate the selection and refinement of trading strategies based on empirical data and visual insights, it fills the gap that exists between sophisticated quantitative analysis and practical implementation for traders or analysts.

4.5 Chapter Summary and Evaluation

The documentation's Chapter 4 offers a thorough analysis of the system architecture for trading strategy optimization, which is based on the intelligent use of Bayesian Optimisation. This method's integration with Gaussian Processes to effectively represent and optimize complex objective functions is explained in detail. The chapter goes into detail on a number of elements, such as the acquisition functions—such as Probability of Improvement and Expected Improvement—which direct sampling in a way that efficiently strikes a balance between exploration and exploitation. It also goes over the full project workflow, highlighting how each step helps to create and improve reliable trading methods, from data collecting and preprocessing to real-world testing.

The final section of the chapter concentrates on the layout and operation of the user interface, which includes dynamic visualisations such as 3D graphs to improve user interaction with the system. Through this interface, users can choose from a variety of trading strategies, alter various parameters, and see a visual representation of how these changes affect the strategy's performance. Because of the thorough setup, users may actively participate in the optimisation process and make strategic decisions based on quantitative data and visual insights. This chapter shows how the project can bridge the gap between sophisticated quantitative analysis and approachable trading solutions by tying together intricate technical procedures with real-world applications.

Chapter 5

Implementation and Testing

5.0 Implementation and Result

This chapter describes the trading strategies optimization process's execution and outcomes. Implementation and Results are the two main sections that make up this chapter. The logic of the approach and the procedures used to collect and preprocess the data are described in the section on implementation. The strategy's performance evaluation, along with the optimal parameters found through optimization and the accompanying performance indicators like the Sharpe Ratio, average return, and maximum drawdown, are shown in the Results section. The results are discussed in the last section, which also includes a 3D model that illustrates the parameter optimization procedure.

5.1 Implementation

5.1.1 Data Extraction

```
# Initialize the exchange
exchange = ccxt.binance()

# Parameters
symbol = 'BTC/USDT'
timeframe = '1h'
since = exchange.parse8601('2020-05-11T00:00:00Z')
now = exchange.milliseconds()

# List to hold fetched data
all_ohlcv = []

# Fetch data in chunks
while since < now:
    try:
        ohlcv = exchange.fetch_ohlcv(symbol, timeframe, since)
        if not ohlcv:
            break
        all_ohlcv += ohlcv
        # Update since to the last timestamp + 1 millisecond to avoid duplication
        since = ohlcv[-1][0] + 1
    except ccxt.NetworkError as e:
        print(f'NetworkError: {str(e)}. Retrying...')
    except ccxt.ExchangeError as e:
        print(f'ExchangeError: {str(e)}. Retrying...')
    except Exception as e:
        print(f'Error: {str(e)}. Stopping...')
        break

# Convert to DataFrame
df = pd.DataFrame(all_ohlcv, columns=['Date', 'Open', 'High', 'Low', 'Close', 'Volume'])
df['Date'] = pd.to_datetime(df['Date'], unit='ms')
```

Figure 5.1.1 Data Extraction Code

Using the ccxt library, the data extraction method retrieves historical OHLCV (Open, High, Low, Close, Volume) data for Bitcoin (BTC/USDT) from the Binance exchange. The information is obtained every hour beginning on May 11, 2020, and continuing until the present. The data is fetched in batches and looped throughout the whole time range because the Binance API has a limit on the amount of data that can be retrieved in a single request. For additional investigation, the data is subsequently transformed into a Pandas DataFrame.

5.1.2 Data Preprocessing

```
def check_and_fill_nulls(data):  
    # Check for null values  
    null_counts = data.isnull().sum()  
    print("Null values in each column before filling:")  
    print(null_counts)  
  
    # If there are any null values, perform Forward-Fill  
    if null_counts.any():  
        data.fillna(method='ffill', inplace=True)  
        print("\nPerformed Forward-Fill to handle null values.")  
  
    # Check for null values again after filling  
    null_counts_after_fill = data.isnull().sum()  
    print("\nNull values in each column after filling:")  
    print(null_counts_after_fill)  
  
    return data
```

Figure 5.1.2 Data Preprocessing Code

The preprocessing stage is essential to guaranteeing the data's integrity. In this stage, we search the dataset for any missing values and use the forward-fill technique to fill them in. By using this technique, the continuity of the dataset is preserved and any gaps in the data are filled in with the prior valid values. Subsequent processes, including computing the moving average and executing the trading strategy, can now be performed on the preprocessed data.

5.1.3 Strategy Deployment

```
def macd_strategy(data, fast_length=12, slow_length=26):
    # Define a fixed macd_length
    macd_length = 5

    # Calculate the MACD and its signal line
    data['MACD'] = ta.trend.ema_indicator(data['Close'], window=fast_length) - ta.trend.ema_indicator(data['Close'], window=slow_length)
    data['Signal'] = ta.trend.ema_indicator(data['MACD'], window=macd_length)
    data['Delta'] = data['MACD'] - data['Signal']

    # Generate signals
    data['Signal_Long'] = np.where((data['Delta'].shift(1) < 0) & (data['Delta'] > 0), 1, 0)
    data['Signal_Short'] = np.where((data['Delta'].shift(1) > 0) & (data['Delta'] < 0), -1, 0)

    data['Position (Long Short)'] = data['Signal_Long'] + data['Signal_Short']

    # Define trading fees
    taker_fee = 0.00055 # 0.055%

    # Calculate PnL with fees, Cumulative PnL, and Drawdown
    data['PnL (Long Short)'] = data['Position (Long Short)'].shift(1) * (data['Close'] / data['Close'].shift(1) - 1)

    # Determine if the trade was a maker or taker (for simplicity, we'll assume all trades are taker trades)
    data['PnL (Long Short) with Fees'] = data['PnL (Long Short)'] - (data['Position (Long Short)'].shift(1) * taker_fee)

    data['Cumulative PnL (Long Short)'] = data['PnL (Long Short) with Fees'].cumsum()
    data['Cumulative Max'] = data['Cumulative PnL (Long Short)'].cummax()
    data['Drawdown (Long Short)'] = data['Cumulative Max'] - data['Cumulative PnL (Long Short)']

    # Calculate Number of Trades
    data['Number of Trades'] = ((data['Position (Long Short)'] != 0) & (data['Position (Long Short)'] != data['Position (Long Short)'].shift(1))).cumsum()

    return data
```

Figure 5.1.3 MACD Strategy code

Two varying-length exponential moving averages (EMAs) are used in the MACD technique to monitor price momentum. When it comes to price fluctuations, the slower EMA (longer period) offers a more steady, smoother picture of the price trend than the faster EMA (shorter duration). An EMA of the MACD values over a given length is used to produce the Signal line, which is a smoothed form of the MACD. The MACD line is computed by subtracting the slow EMA from the fast EMA.

By comparing the MACD and Signal lines, the method produces buy and sell signals. When the MACD crosses above the Signal line, suggesting potential upward momentum, a long signal is generated; when the MACD crosses below the Signal line, suggesting potential downward momentum, a short signal is generated. The Signal_Long and Signal_Short variables are used to record these signals. These signals are then merged to create a single trading position (Position (Long Short)), which indicates if a long, short, or no position is held at any one time.

Furthermore, the strategy uses a fixed taker charge of 0.055% to account for trading fees in its computations. In order to give a more accurate indication of the strategy's profitability after transaction costs, this fee is subtracted from the PnL. The PnL is calculated by taking the percentage change in the closing price between consecutive time steps and multiplying it

by the previous trading position. The cumulative PnL, or running total of the strategy's earnings and losses over time, is then computed using the adjusted PnL (allowing for fees).

The drawdown, which calculates the fall from a high in the cumulative PnL, is another significant parameter determined during preprocessing. The drawdown illustrates how much the performance has dropped from its peak, offering information into the risk and volatility of the approach. The difference between the cumulative maximum PnL and the current cumulative PnL at any given point is used to compute the drawdown.

Lastly, to get a general idea of trading activity and strategy frequency of trades, the total number of trades completed by the strategy is computed by counting the number of times the trading position switches from long to short (or vice versa).

5.1.4 Parameter Optimizaiton

```
# Define the search space for Bayesian optimization (MACD strategy)
space = [Integer(5, 31, name='fast_length'),
          Integer(20, 51, name='slow_length')]

# Define the objective function to minimize (negative Sharpe Ratio)
@use_named_args(space)
def objective(**params):
    result = macd_strategy(data.copy(), **params)
    metrics = performance_metrics(result, timeframe)
    if np.isnan(metrics['Sharpe Ratio']):
        return 0
    return -metrics['Sharpe Ratio']

# Perform Bayesian optimization
res = gp_minimize(objective, space, n_calls=50, random_state=0)

# Get the best parameters and the corresponding metrics
macd_best_params = {dim.name: res.x[i] for i, dim in enumerate(space)}
macd_best_result = macd_strategy(data, **macd_best_params)
macd_best_metrics = performance_metrics(macd_best_result, timeframe)
```

Figure 5.1.4 Bayesian Optimization Code

The first step in the optimization procedure is to define a search space for the two important parameters. While the `slow_length` spans from 20 to 51, the `fast_length` is fixed to fluctuate between 5 and 31. With the help of these ranges, the optimization algorithm can investigate different pairings of the two parameters to find the optimum performing pair.

To determine the Sharpe Ratio for any set of parameters, an objective function is developed. The current parameter values are used to apply the MACD technique, and performance metrics such as the Sharpe Ratio are assessed. Because the optimization method aims to minimize the objective function, the function yields the negative of the Sharpe Ratio. As a result, maximizing the Sharpe Ratio is identical to minimizing the negative Sharpe Ratio. To prevent optimization problems, the function returns 0 if the Sharpe Ratio is invalid (NaN).

The `gp_minimize` function, which minimizes the objective function over the specified search space, is used to carry out Bayesian optimization. In order to identify the ideal pair, the optimizer evaluates 50 alternative parameter combinations over the course of 50 iterations. To guarantee that the outcomes may be repeated, a random seed is set during the process.

After optimization is finished, the optimal `fast_length` and `slow_length` parameter values are extracted, and the MACD strategy is reapplied with these optimized values. The strategy's ultimate performance is then evaluated by recalculating the performance metrics. By increasing the Sharpe Ratio, this procedure aids in optimizing the MACD strategy's profitability and lowering its risk.

```
from sklearn.model_selection import ParameterGrid

# Define parameter grid for Grid Search
param_grid = {
    'fast_length': list(range(5, 31, 5)),
    'slow_length': list(np.arange(20, 51, 5))
}

# Run Grid Search
results = []
for params in ParameterGrid(param_grid):
    result = macd_strategy(data.copy(), **params)
    metrics = performance_metrics(result, timeframe)
    sharpe_ratio = metrics['Sharpe Ratio']
    if not np.isnan(sharpe_ratio):
        results.append((params, sharpe_ratio))
    else:
        print(f"Skipping params {params} due to NaN Sharpe Ratio")
```

Figure 5.1.5 Grid Search CV Code

Both `fast_length` (which ranges from 5 to 31 in steps of 5) and `slow_length` (which ranges from 20 to 51 in steps of 5) have parameter grids defined in the code. This makes sure that during the search, all possible combinations of these two criteria are taken into account. The MACD technique is applied with the specified parameters for each combination, and metrics like the Sharpe Ratio are calculated to assess the strategy's effectiveness. The parameter combination and matching Sharpe Ratio are saved in the results list if the Sharpe Ratio is valid.

Grid Search evaluates every possible combination of parameters, regardless of how well they perform, making it comprehensive yet potentially computationally demanding. This guarantees that no possible combination goes unnoticed and offers a thorough examination of how various parameter pairings impact the strategy's effectiveness. To prevent errors in the analysis, certain combinations are skipped if they yield erroneous findings (such a NaN Sharpe Ratio). A list of all parameter combinations and the corresponding Sharpe Ratios is the ultimate result of the Grid Search procedure, from which the pair that performs the best can be determined.

5.1.5 3D Visualization

```
# Extract the evaluated points and corresponding negative Sharpe Ratios
x_vals = [point[0] for point in res.x_iters]
y_vals = [point[1] for point in res.x_iters]
z_vals = res.func_vals

# Convert negative Sharpe Ratios to positive for visualization
sharpe_ratios = [-z for z in z_vals]

# Find the highest Sharpe Ratio and its corresponding parameters
max_sharpe_ratio = max(sharpe_ratios)
max_index = sharpe_ratios.index(max_sharpe_ratio)
best_fast_length = x_vals[max_index]
best_slow_length = y_vals[max_index]

# Create a 3D scatter plot
fig = go.Figure(data=[go.Scatter3d(
    x=x_vals,
    y=y_vals,
    z=sharpe_ratios,
    mode='markers',
    marker=dict(
        size=5,
        color=sharpe_ratios, # Color by Sharpe Ratio
        colorscale='Viridis',
        opacity=0.8,
        colorbar=dict(title='Sharpe Ratio')
    ),
    text=[f'Fast Length: {x}, Slow Length: {y}, Sharpe Ratio: {z:.4f}' for x, y, z in zip(x_vals, y_vals, sharpe_ratios)],
    hoverinfo='text'
)])

# Highlight the highest Sharpe Ratio point
fig.add_trace(go.Scatter3d(
    x=[best_fast_length],
    y=[best_slow_length],
    z=[max_sharpe_ratio],
    mode='markers+text',
    marker=dict(
        size=7,
        color='red'
    ),
    text=[f'Max Sharpe Ratio: {max_sharpe_ratio:.4f}'],
    textposition='top center'
))

# Set Labels and title
fig.update_layout(
    scene=dict(
        xaxis_title='Fast Length',
        yaxis_title='Slow Length',
        zaxis_title='Sharpe Ratio'
    ),
    title='3D Visualization of Bayesian Optimization for MACD Strategy',
    autosize=True
)

fig.show()
```

Figure 5.1.6 Visualization code of Bayesian Optimization

The plot offers a graphic representation of the relationship between the Sharpe Ratio and various combinations of the fast and slow EMA lengths. The 3D plot's z axis shows the Sharpe Ratio, and the x and y axes show the values of fast_length and slow_length, respectively. A particular combination of these parameters that was assessed during the optimization process is represented by each point in the plot. Warmer colors correspond to

higher Sharpe Ratios, and the color of each point, which is mapped using the Viridis color scale, gives the Sharpe Ratio for that combination of parameters. Additionally, the graphic displays the optimal combination of parameters that produces the highest Sharpe Ratio. It is simple to visually identify this point because it is labeled with the Sharpe Ratio value and is marked in red. When hovered over, the hover text functionality offers further details by revealing the precise fast_length, slow_length, and Sharpe Ratio values for each point.

```
# Ensure results were populated
if not results:
    print("No valid results found. Check strategy implementation and data.")
else:
    # Extract parameter values and Sharpe Ratios
    x_vals = [params['fast_length'] for params, _ in results]
    y_vals = [params['slow_length'] for params, _ in results]
    z_vals = [sharpe for _, sharpe in results]

    # Create grid for surface plot
    x_grid, y_grid = np.meshgrid(np.unique(x_vals), np.unique(y_vals))
    z_grid = np.zeros_like(x_grid, dtype=float)

    # Populate the z values based on the unique x and y values
    for i, x in enumerate(np.unique(x_vals)):
        for j, y in enumerate(np.unique(y_vals)):
            sharpe_values = [z for (xi, yi), z in zip(zip(x_vals, y_vals), z_vals) if xi == x and yi == y]
            if sharpe_values:
                z_grid[j, i] = max(sharpe_values)

    # Find the highest Sharpe Ratio
    max_sharpe_idx = np.unravel_index(np.argmax(z_grid, axis=None), z_grid.shape)
    max_sharpe_value = z_grid[max_sharpe_idx]
    best_fast_length = x_grid[max_sharpe_idx]
    best_slow_length = y_grid[max_sharpe_idx]

    # Plotly 3D surface plot
    fig = go.Figure(data=[go.Surface(x=x_grid, y=y_grid, z=z_grid, colorscale='Viridis')])

    # Add a scatter point for the maximum Sharpe Ratio
    fig.add_trace(go.Scatter3d(
        x=[best_fast_length],
        y=[best_slow_length],
        z=[max_sharpe_value],
        mode='markers+text',
        marker=dict(size=5, color='red'),
        text=[f'Max Sharpe Ratio: {max_sharpe_value:.4f}'],
        textposition='top center'
    ))

    fig.update_layout(title='3D Surface Plot for MACD Strategy (Grid Search CV)', scene=dict(
        xaxis_title='Fast Length',
        yaxis_title='Slow Length',
        zaxis_title='Sharpe Ratio'
    ))
    fig.show()
```

Figure 5.1.7 Visualization code of Grid Search CV

Using a grid of potential parameter combinations, the code assesses each pair and determines the associated Sharpe Ratio, which is subsequently utilized to fill a surface plot. The plot's fast_length and slow_length values are represented by the x and y axes, and the Sharpe Ratio for each combination is displayed on the z axis. The Viridis color scale is used to color the surface; warmer hues correspond to higher Sharpe Ratios. It is simple to observe how different parameter selections impact the strategy's performance with this visual representation, which also shows which parameter combination yields the highest Sharpe Ratio.

The point that corresponds to the maximum Sharpe Ratio is labeled directly on the plot and highlighted in red to further improve the visualization. This indicates which combination of slow_length and fast_length is ideal. After reapplying the MACD method to this ideal pair, performance measures including average return, maximum drawdown, and number of trades are reported.

5.1.6 Performance Metrics

```
# Define a dictionary that maps each timeframe to the number of periods per year
timeframe_mapping = {
    "1Min": 365 * 24 * 60,
    "5Min": 365 * 24 * 60 / 5,
    "15Min": 365 * 24 * 60 / 15,
    "30Min": 365 * 24 * 60 / 30,
    "45Min": 365 * 24 * 60 / 45,
    "1H": 365 * 24,
    "4H": 365 * 24 / 4,
    "8H": 365 * 24 / 8,
    "12H": 365 * 24 / 12,
    "1Day": 365,
    "1Week": 365 / 7,
    "1Month": 12
}

#since we are using the 1hour bitcoin data now, so we initialize the timeframe with the value "1H"
timeframe = "1H"
```

Figure 5.1.8 Timeframe Mapping Code

```
def performance_metrics(data, timeframe):
    # Look up the number of periods per year based on the timeframe
    periods_per_year = timeframe_mapping.get(timeframe, None)

    if periods_per_year is None:
        raise ValueError("Invalid timeframe. Please choose from the following: " + ", ".join(timeframe_mapping.keys()))

    # Sharpe Ratio calculation
    sharpe_ratio = (data['PnL (Long Short)'].mean() /
                    data['PnL (Long Short)'].std()) * np.sqrt(periods_per_year)

    # Average Return calculation
    average_return = data['PnL (Long Short)'].mean() * periods_per_year

    # Maximum Drawdown calculation
    maximum_drawdown = data['Drawdown (Long Short)'].max()

    # Calmar Ratio calculation
    calmar_ratio = average_return / abs(maximum_drawdown) if maximum_drawdown != 0 else np.nan

    # Number of Trades
    number_of_trades = data['Number of Trades'].max()

    return {
        'Sharpe Ratio': sharpe_ratio,
        'Average Return': average_return * 100,
        'Maximum Drawdown': maximum_drawdown * 100,
        'Calmar Ratio': calmar_ratio,
        'Number of Trades': number_of_trades
    }
```

Figure 5.1.9 Performance Metrics Code

Sharpe Ratio, average return, maximum drawdown, Calmar Ratio, and number of trades are among the important performance indicators for the MACD trading strategy that are computed using the given code. These measurements shed light on the trading strategy's risk and efficacy over a specified period of time.

1. **Timeframe Mapping:** The amount of periods each year for various time intervals, such as 1-minute, 1-hour, or 1-day data, is defined by the `timeframe_mapping` dictionary. To standardize the calculations, this mapping is utilized to translate each timeframe into an equivalent amount of periods every year. For instance, there are 365 days in a year and $365 * 24$ periods in a year for hourly data (1H).
2. **Performance Metrics Function:** Based on the input data and the chosen timeframe, the `performance_metrics` function determines how well the strategy performs. The function uses the `timeframe_mapping` dictionary to first determine the number of periods in a year. This allows it to compute annualized metrics like the Sharpe Ratio and average return.
3. **Sharpe Ratio Calculation:** The strategy's risk-adjusted return is measured by the Sharpe Ratio. The computation involves dividing the average daily return by the standard deviation of daily returns, and then squaring the result to get the annualized

return for the number of periods in a year. The excess return generated by the strategy per unit of risk (standard deviation) is shown by this ratio.

4. **Average Return Calculation:** The average return is determined by multiplying the strategy's mean daily return by the number of periods in a year in order to annualize it. This gives an annualized estimate of the predicted return on the approach.
5. **Maximum Drawdown Calculation:** From a peak to the lowest point in the cumulative PnL (Profit and Loss), the maximum drawdown represents the biggest percentage fall. Determining the maximum loss an investor could incur over a specified time frame aids in evaluating the strategy's risk.
6. **Calmar Ratio Calculation:** The year-over-year average return divided by the biggest drawdown's absolute value is known as the Calmar Ratio. The strategy's performance is measured in relation to the worst observed drawdown, and it offers a risk-adjusted return metric.
7. **Number of Trades Calculation:** Calculated as the greatest value in the Number of Trades column, the number of trades shows the total number of trades executed by the strategy. A summary of the strategy's trading frequency over the studied time is provided by this.
8. **Return of Performance Metrics:** The function returns a dictionary containing all these performance metrics: Sharpe Ratio, average return, maximum drawdown, Calmar Ratio, and the number of trades. These metrics help assess the overall performance, profitability, and risk profile of the MACD strategy over the selected timeframe.

5.2 Result

5.2.1 Grid Search CV Best Parameter Combinations

Trading Strategy	Parameter 1	Parameter 2
MACD (Moving Average Convergence/Divergence) Strategy	fast_length = 15	slow_length = 45
Moving Average 2 Line Cross Strategy	fast_length = 10	slow_length = 30
ADX DI Crossover Strategy(Momentum BreakOut)	adx_length = 15	adx_threshold = 40
Mean Reversion	move_limit = 3	ma_length = 50
Bollinger band + RSI	rsi_length = 100	bb_length = 29
Z_score_sma	sma_length = 60	z_score_threshold = 3.0000

Table 5.2.1 Best Parameter Combinations of Grid Search CV

5.2.2 Bayesian Optimization Best Parameter Combinations

Trading Strategy	Parameter 1	Parameter 2
MACD	fast_length = 20	slow_length = 37
Moving Average 2 Line Cross Strategy	fast_length = 12	slow_length = 23
ADX DI Crossover Strategy	adx_length = 51	adx_threshold = 12
Mean Reversion	move_limit = 20.1791	ma_length = 118
Bollinger band + RSI	rsi_length = 101	bb_length = 15
Z_score_sma	sma_length = 71	z_score_threshold = 2.4614

Table 5.2.2 Best Parameter Combinations of Bayesian Optimization

5.2.3 Evaluation of Grid Search CV

Trading Strategy	Sharpe Ratio	Average Return(%)	Maximum Drawdown(%)	Calmar Ratio	Number of Trades
MACD	0.7377	13.3495	19.2247	0.6944	3020
Moving Average 2 Line Cross Strategy	0.7482	47.0734	124.4974	0.37811	1546
ADX DI Crossover Strategy	1.9099	120.0111	68.3515	1.7558	31
Mean Reversion	1.0920	68.7843	159.3777	0.4316	2142
Bollinger band + RSI	0.3367091876 3834934	9.9880709 1157666	62.057045459 90499	0.1609	2
Z_score_sma	1.4061	88.2961	58.0140	1.5220	144

Table 5.2.3 Evaluation of Grid Search CV result

5.2.4 Evaluation of Bayesian Optimization

Trading Strategy	Sharpe Ratio	Average Return(%)	Maximum Drawdown(%)	Calmar Ratio	Number of Trades
MACD	0.8625	15.6099	20.6430	0.7562	2794
Moving Average 2 Line Cross Strategy	0.9140	57.5116	155.1906	0.37059	1933
ADX DI Crossover Strategy	1.0632	66.8049	77.4744	0.8623	511
Mean Reversion	1.3581	85.5371	95.3308	0.8973	1392
Bollinger band + RSI	0.3367	9.9882	62.0570	0.1609	2
Z_score_sma	1.2270	77.1634	67.4776	1.1435	206

Table 5.2.4 Evaluation of Bayesian Optimization result

5.3 Chapter Summary and Evaluation

The application, optimization, and assessment of the MACD trading technique are covered in detail in Chapter 5. Following the strategy deployment and parameter optimization utilizing both Grid Search and Bayesian Optimization approaches, it covers data extraction and preparation. In addition to performance metrics like average return, maximum drawdown, and number of trades being generated, a 3D graphic shows how different parameter combinations affect the Sharpe Ratio. In order to better understand how each method can increase the risk-adjusted returns of the strategy, a comparative study of Grid Search and Bayesian Optimization is presented in the next chapter.

Chapter 6

Deployment and Real World Testing

6.0 Deployment and Real World Testing

In Chapter 6, the trading strategy optimization project is implemented on the Streamlit platform. Users can interact with the program by uploading data, choosing optimization techniques, and viewing the outcomes. It also discusses the practical testing of the "Z_score_sma" approach with two distinct parameter sets derived from Bayesian Optimization and Grid Search CV. Using the "Cybotrade Cloud" platform, two Bybit accounts were used to test the approach in real time.

6.1 Deployment (Streamlit)

The Streamlit platform was utilized in the project's implementation to give traders an intuitive, interactive interface for optimizing their quantitative trading strategies. The interface screenshot of the Streamlit app shows that it has a tidy and user-friendly configuration.

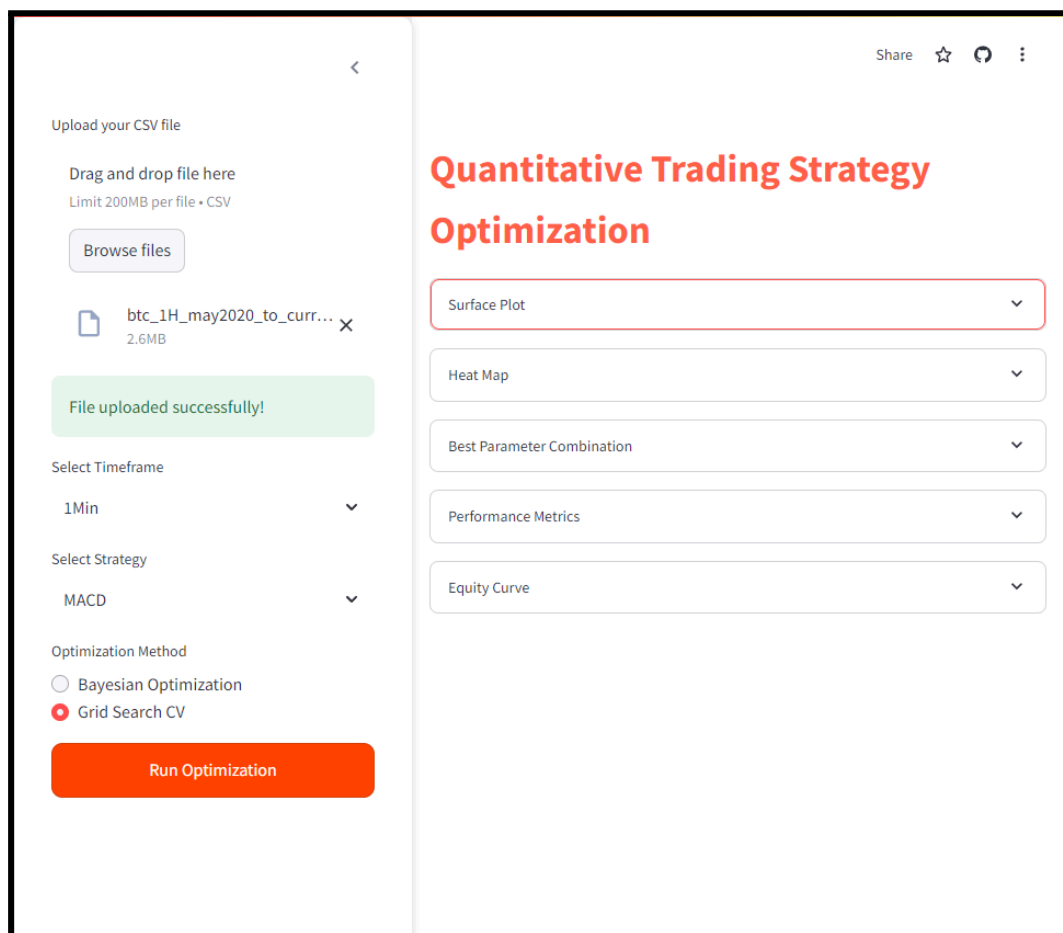


Figure 6.1.1 User Interface for Grid Search CV Optimization Method

Key Features of the Deployment:**1. File Upload:**

- Users can upload a CSV file containing historical data (e.g., Bitcoin price data). The file size is limited to 200MB to ensure smooth data processing.
- Once uploaded, the app confirms successful data ingestion, preparing the system for analysis.

2. Timeframe and Strategy Selection:

- Users can select the desired timeframe, such as 1 minute, for the analysis.
- They can also choose from multiple trading strategies, such as MACD, to perform backtesting and optimization.

3. Optimization Method Selection:

- Users have the flexibility to choose between two optimization methods: .
 1. **Bayesian Optimization** focuses on efficiently exploring the parameter space.
 2. **Grid Search CV** exhaustively searches through the parameter combinations, and it provides access to a **heatmap** feature, which visualizes the performance of different parameter combinations.
- The "Run Optimization" button becomes available after the selection, allowing the user to start the optimization process.

4. Results Display:

- **Surface Plot:** Available for both optimization methods, it provides a 3D visualization of the parameter space and how different combinations affect performance.
- **Heatmap:** Exclusively available for **Grid Search CV**, it visualizes the performance of parameter combinations in a 2D heatmap, offering a quick overview of where the best parameters lie.

- **Best Parameter Combination:** Shows the optimal parameters found by either optimization method.
- **Performance Metrics:** Displays key performance indicators like Sharpe Ratio, average return, and maximum drawdown, giving insights into strategy effectiveness.
- **Equity Curve:** Plots the equity growth over time based on the optimized strategy, helping users visualize profitability.

6.2 Real World Testing

I used Cybotrade Cloud for practical testing to confirm the effectiveness of the Z-score SMA method. This approach was created and implemented using two sets of optimal parameters derived from Grid Search CV and Bayesian Optimization.

6.2.1 Preparation for Testing

Before running the strategy on Cybotrade, four key files were needed:

1. **credentials.json:** The API credentials to use Cybotrade to connect to the Bybit exchange are contained in this file. To guarantee safe access to the trading account, it contains the environment configuration, API key, and API secret.
2. **cybotrade.yaml:** The system configuration is defined in this file, which also makes sure that the exchange settings and environment variables required for Cybotrade's Bybit connection are set up appropriately.
3. **main.py:** The Z-score SMA technique, which is the trading logic, is implemented by this Python script. Using integration with Cybotrade, the script executes live trades according to parameters optimized using Grid Search CV and Bayesian Optimization.

4. **requirements.txt**: The Python prerequisites required to run the strategy on Cybotrade Cloud are listed in this file. It uses programs like pandas, numpy, and cybotrade to manage trading activities and strategy logic.

6.2.2 Strategy Testing

Since the goal of my study is to build efficient trading strategies, I chose the Z-score SMA technique since it is strong in spotting overbought or oversold positions. Two sets of parameters were used to test the strategy:

1. **Bayesian Optimization Parameters:**
 - SMA Length: 90
 - Z-score Threshold: 2.707386458474312
2. **Grid Search CV Parameters:**
 - SMA Length: 60
 - Z-score Threshold: 3.0000000000000004

6.2.3 Creating an Instance on Cybotrade Cloud

Figure 6.1.1 Instance Creation Page On Cybotrade Cloud

Setting up an instance on the Cybotrade Cloud platform to execute the Z-score SMA strategy in a live trading environment came next, when the required files (credentials.json, cybotrade.yaml, main.py, and requirements.txt) were ready.

1. Access Cybotrade Cloud

I used my Discord account to log into the Cybotrade Cloud platform, which offers a simplified setting for creating and executing trading strategies.

2. Create an Instance:

I navigated to the "**Create Instance**" section of the dashboard. In the instance setup screen (as shown in the screenshot), I selected the following options:

- **Name:** A unique name for my instance (eg. Bayesian Optimization and Permutation Testing(Grid Search CV)).

- **Image:** I chose **Python** as the image, as the strategy code is written in Python.
- **Region:** I selected **Singapore** for lower latency, as my trading account is located in that region.
- **Instance Type:** I opted for the **Basic** instance, which charges \$0.04152 per hour, sufficient for the scale of my testing.

3. Upload Files:

Using Cybotrade's interface, I zipped and uploaded the ready files (requirements.txt, main.py, cybotrade.yaml, and credentials.json) to the instance after it had been created. Subsequently, the main.py script was run in the cloud environment, utilizing Cybotrade's infrastructure to implement the strategy instantly.

4. Monitor the Strategy:

While the instance was operating, I kept an eye on the Z-score SMA strategy's performance in real time, tracking performance based on the two sets of parameters that were improved using Grid Search CV and Bayesian Optimization. This included live trade executions.

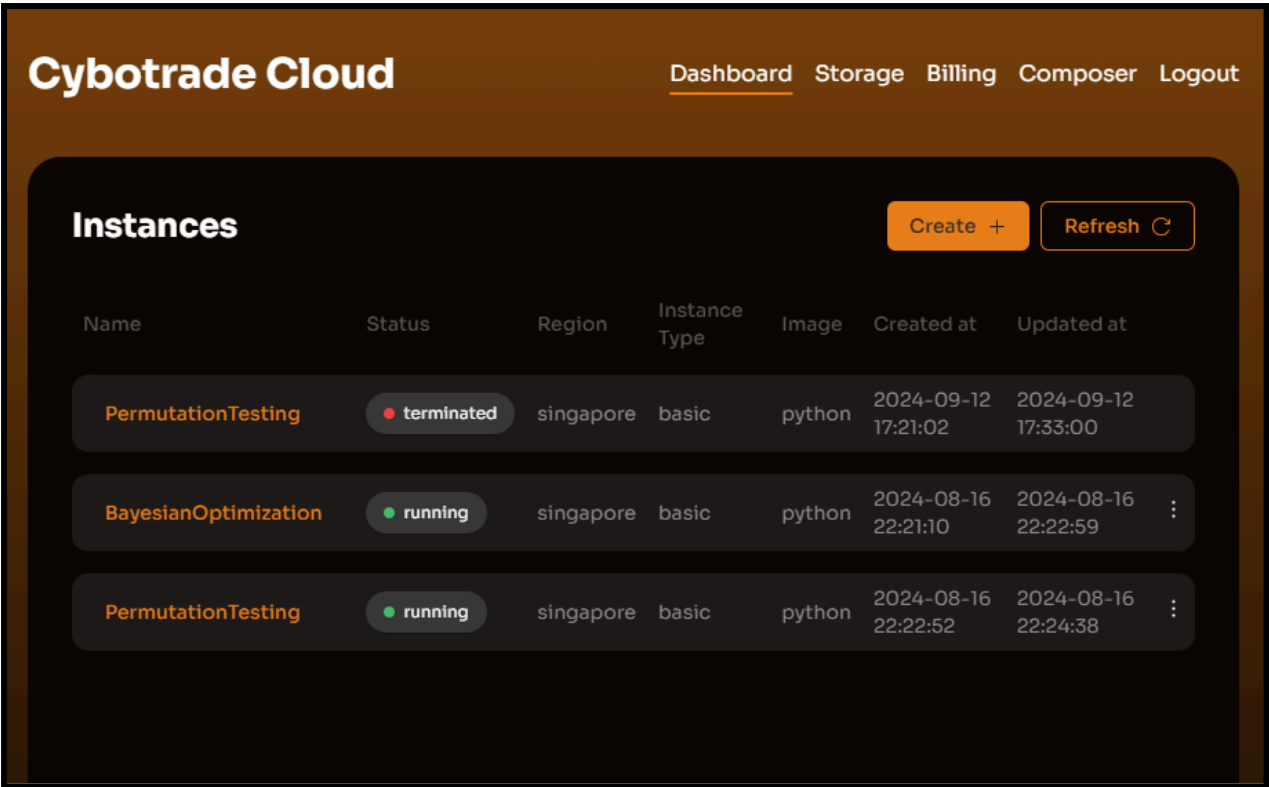


Figure 6.1.1 Dashboard of Cybotrade Cloud

6.2.3 Real-World Testing Results

I implemented the Z-score SMA technique on the Cybotrade Cloud utilizing both Grid Search CV and Bayesian Optimization parameters after creating two distinct Bybit accounts. Tests of both optimization techniques in an actual trading setting and performance evaluations under real-time market conditions were the objectives. Both Bybit accounts were initialized with an initial capital of 100 USD each. The testing period started on 12 August 2024 and is estimated to last for 30 days, concluding on 4 September 2024.

Monitoring Logs and Artifacts

Both algorithms were successfully connected to their corresponding Bybit accounts during the testing process. The logs from the Cybotrade Cloud dashboard demonstrate that both instances were operating error-free, maintaining a constant connection to Bybit, and waiting for favorable market circumstances before initiating any transactions.

- **Bayesian Optimization Instance:** The instance was operating correctly, as evidenced by the log artifacts, with the Cybotrade runtime and Bybit WebSocket connection operating as they should have. Nevertheless, since the market did not fulfill the parameters established by the strategy's Bayesian Optimization parameter set, no orders were made.
- **Grid Search CV Instance:** In a similar vein, the instance maintained its connection to Bybit and continued to function throughout the testing phase. Although the logs indicate a steady connection to the exchange, no trades were made because the Grid Search CV technique was not suited for the current market conditions.

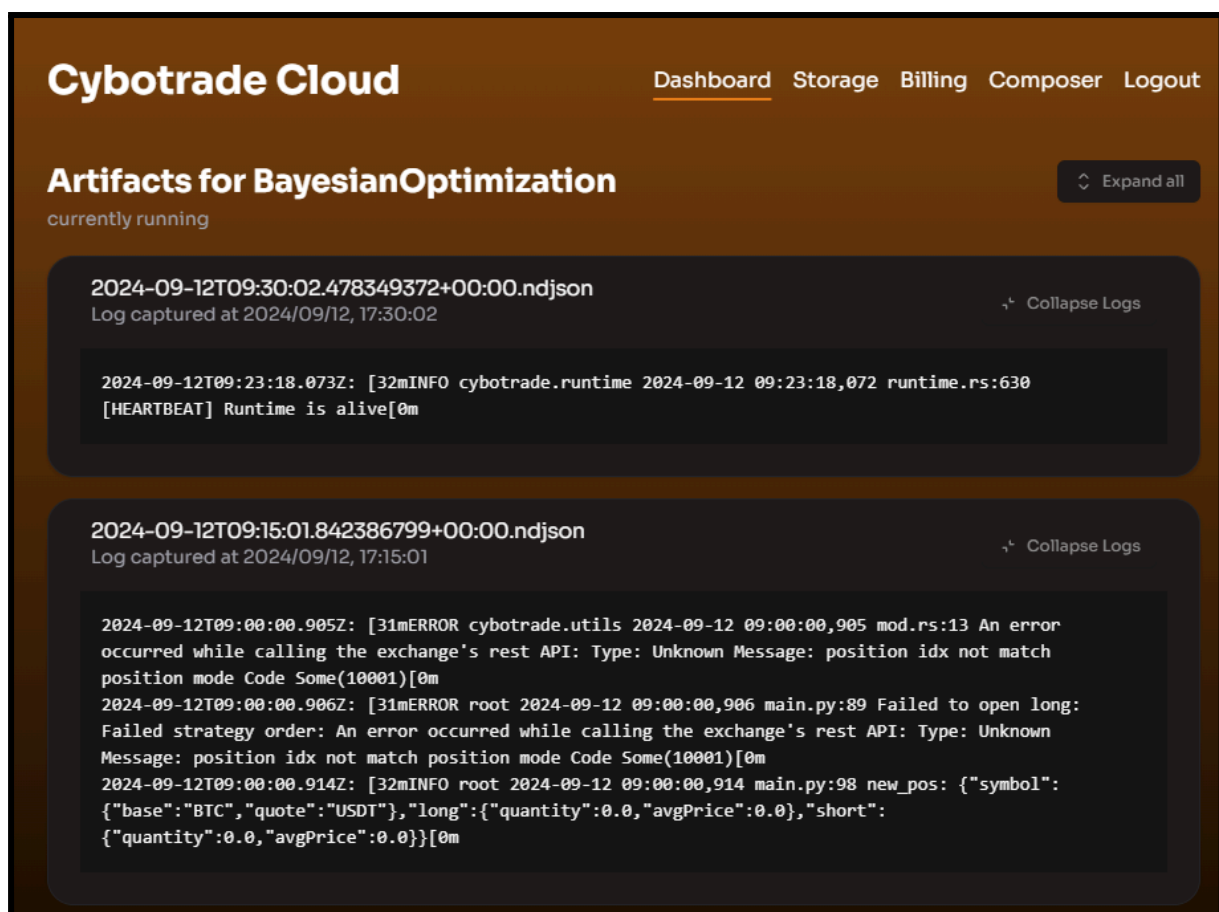


Figure 6.1.1 Log Messages of Bayesian Optimizaition Instance

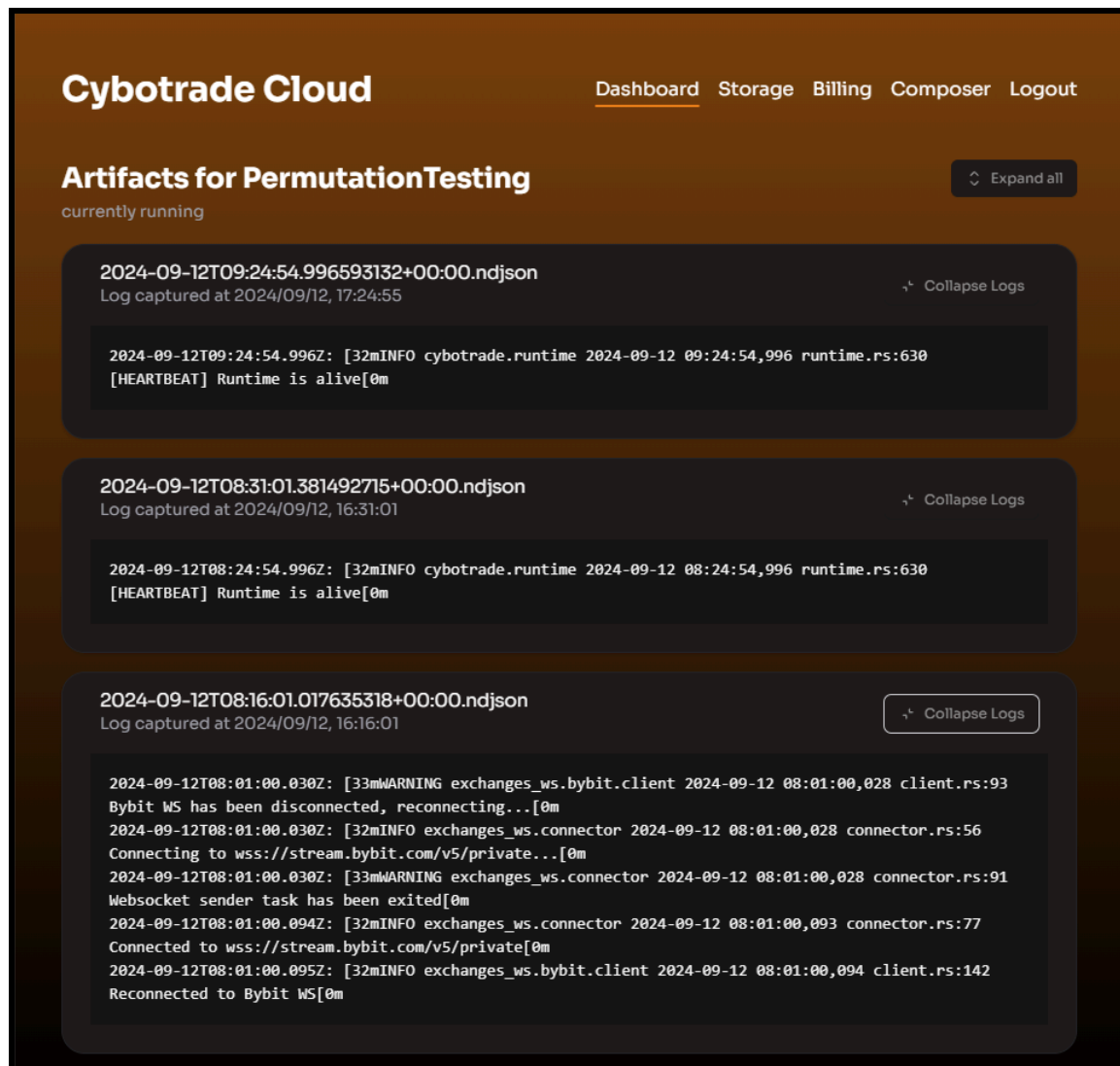


Figure 6.1.1 Log Messages of Grid Search CV Instance

Key Observations

The logs demonstrate that both optimization algorithms were properly applied and connected to the exchange, even if the strategy did not execute any transactions during this first testing phase. The reason for the lack of trade execution is that the Z-score SMA strategy's defined signals are not triggered by the particular market conditions.

- The market conditions required to trigger a trade (based on the Z-score thresholds) were not met during the testing period.
- Both optimization methods are ready for further testing, and trades will be executed once the market aligns with the strategy's conditions.

The significance of market conditions in strategy performance is emphasized by this real-world testing. The testing effectively confirmed the connectivity and system's preparedness to execute transactions when market conditions fit with the strategy, even if the strategies did not place any trades during this time.

6.3 Chapter Summary and Evaluation

The project's deployment and real-world testing were covered in great detail in Chapter 6. The Streamlit platform was used to successfully launch the project, enabling users to communicate with the system, adjust parameters, and view the outcomes instantly. In order to evaluate the Z-score SMA strategy in a real-world setting, Grid Search CV and Bayesian Optimization parameters were used to test the strategy's performance on Cybotrade Cloud.

Problems Faced

The project faced a number of difficulties at this point. Making sure the plan was properly integrated with the Bybit exchange through the Cybotrade platform was one important problem. Even though the strategy was implemented correctly, the market did not match the predetermined requirements for trade execution during the first testing phase, which resulted in no transactions being completed. Another difficult part of the deployment process was managing the setup of the cloud environment and working with massive datasets during optimization.

Solutions

The artifact logs from the Cybotrade Cloud were used for rigorous testing and monitoring in order to resolve the integration problems. Even though trade execution was not possible due to market conditions, the logs verified that the strategy was correctly connected to Bybit and that the system was operating as planned. To make sure the plan works as intended when market circumstances match its requirements, more testing will be done. By making sure that data was correctly packed and uploaded to the Cybotrade platform for effective execution, problems with data handling were also rectified.

Tools and Techniques

In order to create an intuitive web interface that facilitates parameter tuning and result visualization, the deployment made use of Streamlit. Real-time strategy testing in relation to Bybit was conducted using the Cybotrade Cloud platform. The two main methods used to optimize strategy parameters were Grid Search CV and Bayesian Optimization. These tools were selected due to their scalability, simplicity of deployment, and compatibility with Python libraries that are used for developing trading strategies and data analysis.

Chapter 7

Discussions and Conclusion

7.0 Discussions and Conclusion

7.1 Summary

The project's main goal is to use 3D visualization tools to solve the difficulties associated with developing quantitative trading systems. Conventional methods for trading strategy optimization frequently depend on static techniques that don't give enough information about how different factors interact with one another. By creating an interactive 3D visualization tool that incorporates machine learning methods, particularly Bayesian Optimization, the project offers a novel solution to this problem. With the use of this tool, users can examine the intricate interactions between various parameters and see how adjustments to those parameters affect the strategy's effectiveness. Through the use of Streamlit for deployment and Python for data processing, the project skillfully blends cutting-edge methods with an intuitive user interface, facilitating traders' ability to engage with the data and adjust their strategies in real-time.

To effectively explore a wide parameter space and identify optimal settings with little processing resources, machine learning algorithms like Bayesian Optimization were chosen. In contrast, this method can be computationally expensive compared to more conventional optimization strategies such as Grid Search. The visualization element was essential to the project's success since it gave traders a clear understanding of how various parameter configurations impact the trading approach. The practical application of the technology was further proven by backtesting it using actual Bitcoin data. Overall, the project established an innovative approach for trading strategy optimization through the use of machine learning and visualization tools, in addition to offering a strong solution to a complicated problem.

7.2 Achievements

The main accomplishment of this project is the creation of a 3D visualization tool specifically designed for quantitative trading strategy optimization. The project's goal of giving traders a dynamic, user-friendly interface to investigate how various parameter combinations affect strategy performance is successfully achieved by this tool. The technology helps traders make more informed decisions based on data by letting them rapidly visualize the outcomes and change parameters in real-time. The tool's ability to effectively discover optimal parameter values is further enhanced by the inclusion of sophisticated machine learning algorithms, such as Bayesian Optimization. This is especially crucial in trading, as strategy success can be extremely affected by even slight changes to critical parameters.

One notable accomplishment of the project is its ability to use Streamlit and Python to handle and analyze massive datasets. Since the system was built to interact with real-world Bitcoin data, it frequently handles large-scale, historical price data that is collected frequently. The endeavor effectively employed data chunking strategies and parallel processing to overcome the difficulties posed by processing and displaying such enormous volumes of data. This guarantees that trader backtesting tactics on large datasets won't cause serious performance problems. A seamless, simple user experience with little setup and easy sharing was made possible by the deployment team's decision to use Streamlit, which also guaranteed that the project would remain accessible.

Furthermore, the research was highly successful in verifying the efficacy of its optimization techniques. To adjust the parameters of the method, both Grid Search and Bayesian Optimization were used; however, the latter proved very useful because of its effectiveness in scanning wide parameter spaces. This dual optimization methodology offered comparative insights, demonstrating how several techniques could enhance the performance of a strategy. The practical usability of the technology was subsequently demonstrated by backtesting the derived strategies using historical data. As previously discussed in the chapters, the real-world testing phase demonstrated the built tool's strengths in identifying profitable solutions while limiting risk. This accomplishment demonstrates the tool's practical application, enabling traders to optimize and improve their tactics for higher profits and lower risk.

7.3 Contributions

The project presents a number of significant innovations, most notably in its creative use of machine learning and 3D visualization to optimize trading tactics. One of the most significant achievements is the creation of an interactive platform that enables users to investigate intricate interactions between various factors, facilitating the process of determining the ideal configurations for a trading strategy. This fills in a big hole in conventional optimization methods, where parameter interactions are frequently hard to understand. Traders may now have a greater understanding of how various parameter configurations, like moving averages or signal thresholds, impact the strategy's overall performance, risk, and profitability thanks to the dynamic 3D representation. With the help of this tool, traders may make better decisions and modify their strategies with more certainty and accuracy.

Furthermore, a significant technological advancement is the incorporation of Bayesian Optimization into the program. Bayesian Optimization provides a more effective method of exploring the parameter space than traditional optimization approaches such as Grid Search, which necessitate thorough testing of every possible combination of parameters. As a result, the tool becomes more scalable and faster—especially for intricate tactics involving numerous variables. The project also shows how 3D visualization and optimization algorithms can be used to build a potent system with practical applications in quantitative trading. This technology is very relevant from a marketability standpoint to traders and financial institutions who are searching for sophisticated, yet approachable tools to maximize their strategies. The system's potential to support live trading environments and integrate real-time market data makes it a valuable asset in today's fast-paced financial markets.

7.4 Limitations and Future Improvements

This project's primary limitation is that it uses historical data for backtesting instead of current, real-time market data. Despite the fact that the tool uses past Bitcoin price data to refine trading methods, it is currently unable to integrate real-time market data streams. This makes the instrument less useful in real-time trading situations where quick decisions are needed due to constantly shifting market conditions. The tool's utility is limited to past performance analysis without real-time data integration, which may not necessarily translate

well to future market conditions, particularly in volatile areas like bitcoin trading. o future market conditions, especially in volatile markets like cryptocurrency trading.

The absence of automatic cross-asset validation at this time is another drawback. The purpose of the program is to use historical data to optimize strategies for a particular asset, Bitcoin. Diversified portfolios, which comprise several assets from other markets, such equities, commodities, or other cryptocurrencies, are employed by a lot of traders. Lack of a multi-asset optimization framework limits the tool's adaptability and lessens its capacity to offer information concerning various financial instruments. For a larger spectrum of traders and investors that work in more complex market conditions, the tool's support for various assets would greatly increase its usefulness.

One major enhance for the future would be to incorporate real-time data feeds from exchanges like Binance or Bybit. The program could go beyond backtesting and enable traders to optimize tactics in real-time, adapting to the state of the market, by integrating live data. This would create opportunities for real-time warnings, adaptive methods, and potentially automated trading systems, in addition to enhancing the tool's usefulness for live trading. To further increase the resilience and efficacy of the tactics, real-time data integration could be combined with machine learning models that are constantly learning and adapting to changing market conditions.

The optimization system might be expanded to accommodate multi-asset trading, which would be a significant future development. Redesigning the tool would be necessary to enable traders to optimize and visualize strategies across a range of financial assets, such as equities, commodities, and cryptocurrencies. With the capacity to serve a wider range of traders and institutional investors who work in a variety of marketplaces, this feature would greatly boost the tool's marketability. Additionally, by enabling users to observe how various assets interact in a portfolio context, multi-asset support would allow users to optimize portfolio-level techniques, such as diversification or hedging. This would increase the tool's comprehensiveness and relevance to today's trading requirements.

7.5 Issues and Solutions

Initially, one of the biggest problems with the project was trying to locate a trustworthy source of Bitcoin price information. Many of the datasets that were made public had limitations on their use, were not updated in real time, or were incomplete. Because of this, it was challenging to get the high-quality data required for precise backtesting and trading strategy modification. The validity of the project's conclusions could be compromised by biased results caused by inconsistent or erroneous data.

CCXT (bitcoin eXchange Trading) is a library that I used to fix this issue because it allows you to get bitcoin data directly from exchanges like Binance. In order to guarantee that the dataset was precise and complete, this offered a dependable, real-time source of Bitcoin price data with fine precision (such as hourly or minute-level data). With the help of the CCXT library, data integration into the project was made simple and automated data fetching was made possible, which improved the consistency and efficiency of the optimization and backtesting techniques. The project was able to go on with a strong foundation of high-quality data by utilizing CCXT to effectively solve the issue of data reliability.

The difficulty and performance concerns associated with visualizing the high-dimensional parameter space constituted another crucial issue. Due to an abundance of data points and rendering complexity, the first visualizations of the 3D surface plots for several parameters were slow to generate and challenging to understand. Additionally, showing an excessive number of parameter choices crowded the visualizations, which made it more difficult for users to derive useful conclusions about the effectiveness of the strategy.

In order to solve this problem, I simplified the visualization procedure and decreased the quantity of parameters that were shown at once. The answer was to restrict the combinations that were shown to those that had the greatest Sharpe Ratios, so concentrating on the most important factors, such as the fast and slow moving averages. As a result, rendering required less processing power, and the visualizations were easier to understand and more intuitive. Furthermore, I modified the rendering technique by utilizing libraries designed for high-dimensional charting and more effective data structures, which improved the tool's efficiency without sacrificing the clarity of the visual output.

References

1. López de Prado, M. (2018). Marcos Lopez de Prado. Advances in Financial Machine Learning. 1–393.
2. Strategic Optimization of Parameters in Quantitative Trading | AlinaKhay. (n.d.). Retrieved December 24, 2023, from <https://alinakhay.com/Strategic Optimization of Parameters in Quantitative Trading>
3. Chan, E. P. (2014). Algorithmic Trading: Winning Strategies and Their Rationale. In Algorithmic Trading: Winning Strategies and Their Rationale. <https://doi.org/10.1002/9781118676998>
4. Ernestp.chan. (2021). Quantitative Trading : How to Build Your Own Algorithmic Trading Business.
5. Kissell, R. (2020). Algorithmic Trading Methods. In Academic Press. Academic Press.
6. Zura Kakushadze. (2021). 151 Trading Strategies.
7. Best MACD Trading Strategy (Moving Average Convergence Divergence) | by Quantified Strategies | Medium. (Sep 23, 2023.). Retrieved May 18, 2024, from <https://tradingstrategy.medium.com/a-macd-trading-strategy-e198c6224429>
8. Maximizing Profitability of Moving Average Crossover Trading Strategy: What moving averages to use? | by Facundo Joel Allia Fernandez | Medium. (Apr 13, 2023.). Retrieved May 18, 2024, from <https://medium.com/@facujallia/maximizing-profitability-of-moving-average-crossover-trading-strategy-what-moving-averages-to-use-e5cd70085690>
9. Mean Reversion: Simple Trading Strategies Part 1 | by Auquan | auquan | Medium. (Aug 30, 2017.). Retrieved May 18, 2024, from <https://medium.com/auquan/mean-reversion-simple-trading-strategies-part-1-a18a87c1196a>
10. Quant Trading with Python Using Bollinger Bands | by Judy T Raj | Medium. (Feb 27, 2024.). Retrieved May 18, 2024, from <https://medium.com/@judytraj007/quant-trading-with-python-e334c132e5fb>

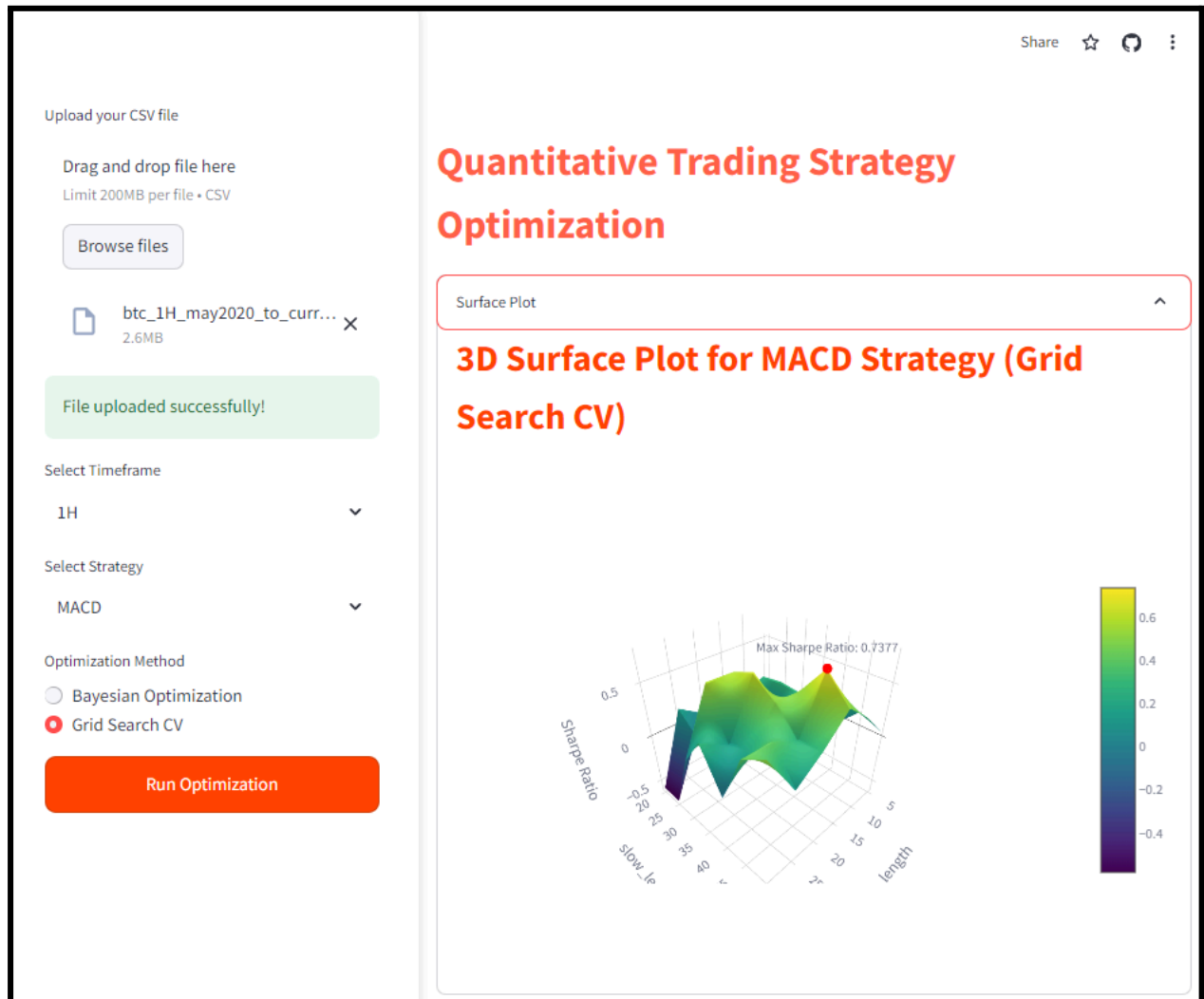
[WFpbCI6ImNoYW5seS13bTlwQHN0dWRlbnQudGFyYy5lZHUubXkiLCJlbWFpbF92ZXJpZmllZCI6dHJ1ZSwibmJmljoxNzI2MjM2MDYwLCJuYW1lIjoITEkgWUFORyBDS](#)
[EFOliwicGljdHVyZSI6Imh0dHBzOi8vbGgzLmdvb2dsZXVzZXJjb250ZW50LmNvbS9h](#)
[L0FDZzhvY0I5cXJTdU5MSlcnNEhELWpZNGVUS0tSWnpuWUI4b1ZXdk2UHpgOW](#)
[pJQTIxXaFJkQT1zOTYtYyIsImdpdmVuX25hbWUiOiJMSSBZQU5HliwiZmFtaWx5X25](#)
[hbWUUiOiJDSEFOliwiaWF0IjoxNzI2MjM2MzYwLCJleHAiOiE3MjYyMzk5NjAsImp0aSI](#)
[6ImIwMDY4MGlyZTFjYTY3NmZmOGVjMmFmOGE4NDc3NGQwMGQ4YjdIMWEifQ](#)
[.WX9IRjGAYJ1pLf26TUIOyFKViZnsT6gTHWDIGu8PH_pmlhEeiP9tVtC4NZNJTIbRh](#)
[MaixLGPboyofSwaCcwONs4bPQk-qtHhZC0FdWHPV2V8OjH5ENZz2qVMYgXyXW](#)
[E21_QUS6Pttobv5bUwRjINys-3q9odQ7_Vg_vSZMAUe6Z4TVGKfMB0TtbGvSnB5E](#)
[wMwN8MiYFMF-WyFLuCKHQ6Ag7eqoSGa7bD6LJxUnR8pbuoSGEfV27fw4fIYHmlq](#)
[aYIhjSE5kGvaBocGTNIYOMdXD00FyfNEO8Vf0hQUsmIG5uGHajXuZntR5aLTnv4W](#)
[Fmwel83OV03XSYypuzZMDBF6w](#)

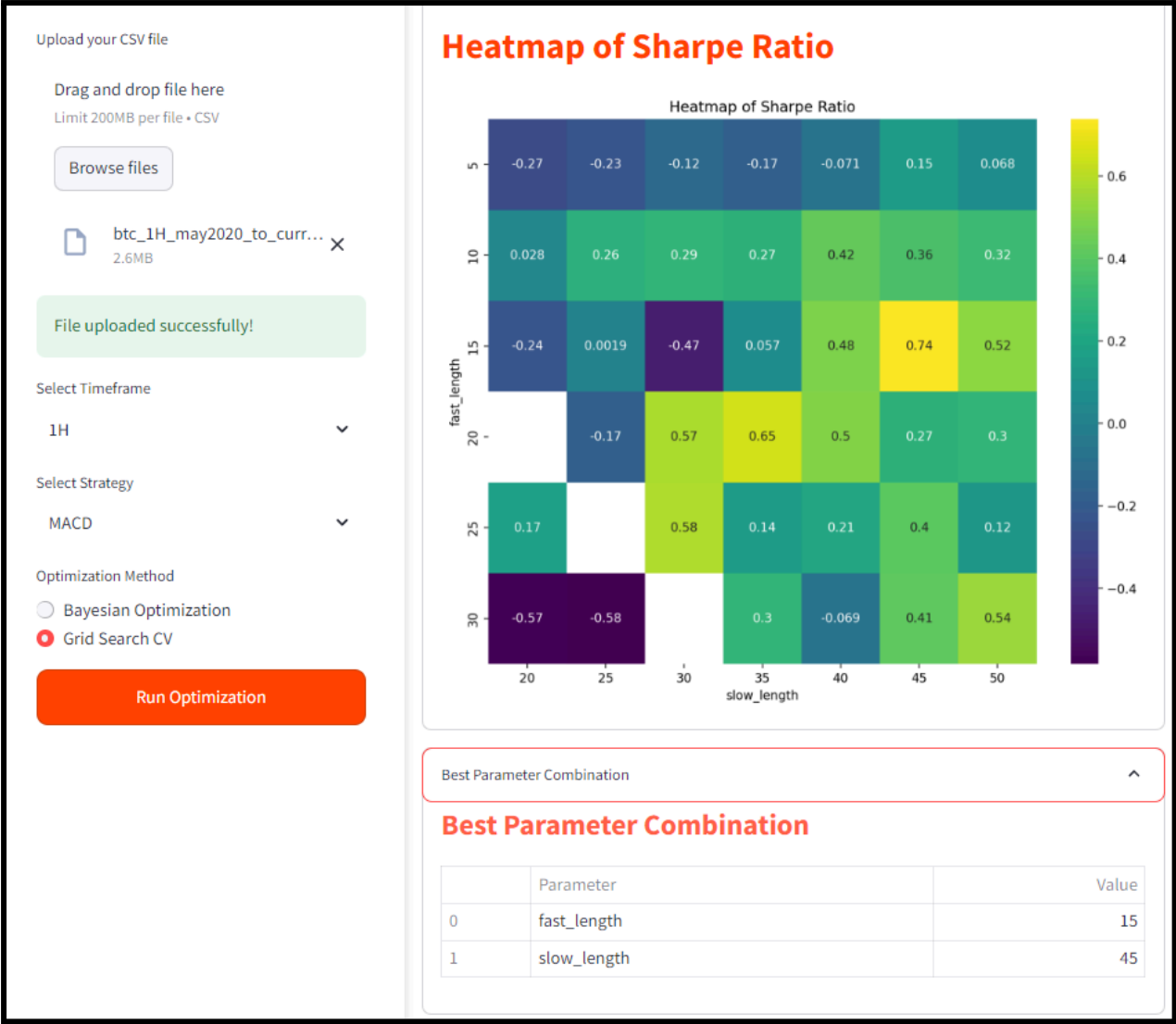
19. Rahul. (2024, August 13). Tune Hyperparameters with GridSearchCV - Analytics Vidhya.

<https://www.analyticsvidhya.com/blog/2021/06/tune-hyperparameters-with-gridsearch-cv/>

Appendix

Appendix 1: User interface of Grid Search CV:





Upload your CSV file

Drag and drop file here

Limit 200MB per file • CSV

Browse files

btc_1H_may2020_to_curr...

2.6MB

File uploaded successfully!

Select Timeframe

1H

Select Strategy

MACD

Optimization Method

Bayesian Optimization

Grid Search CV

Run Optimization

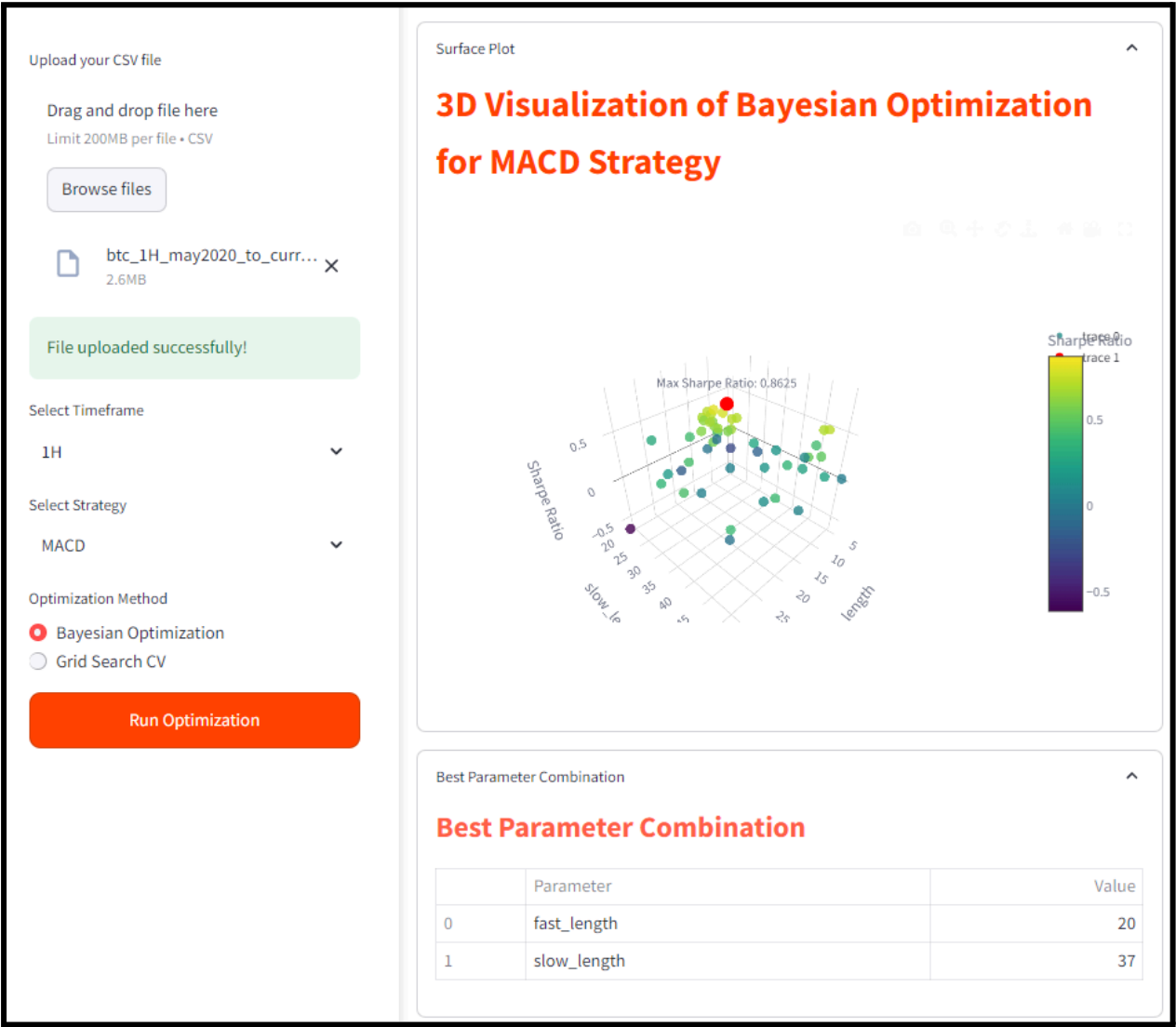
Performance Metrics

Metric	Value
Sharpe Ratio	0.7377
Average Return	13.35%
Maximum Drawdown	19.22%
Calmar Ratio	0.6944
Number of Trades	3020

Equity Curve

MACD Strategy Cumulative PnL

Appendix 2: User interface of Bayesian Optimization :



Appendix 3 : Bybit Account of Bayesian Optimization Implementation

BYBIT

ExchangeWEB3Buy CryptoMarketsTradeDerivativesToolsFinanceNEIRO/USD

DepositAssetsOrders

TransferDepositConvertLoanHistoryOrders

Derivatives

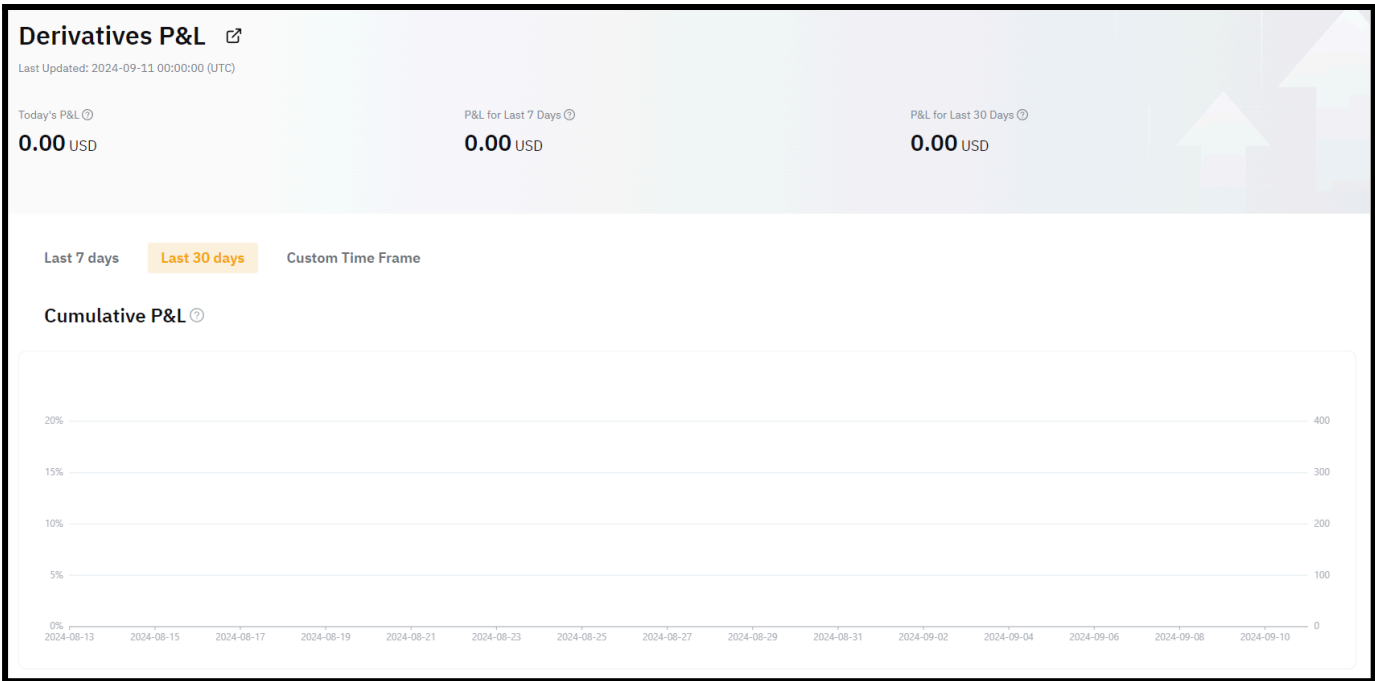
Total Equity100.00 USD0.00172070 BTC

Unrealized P&L0.00 USD0.00000000 BTC

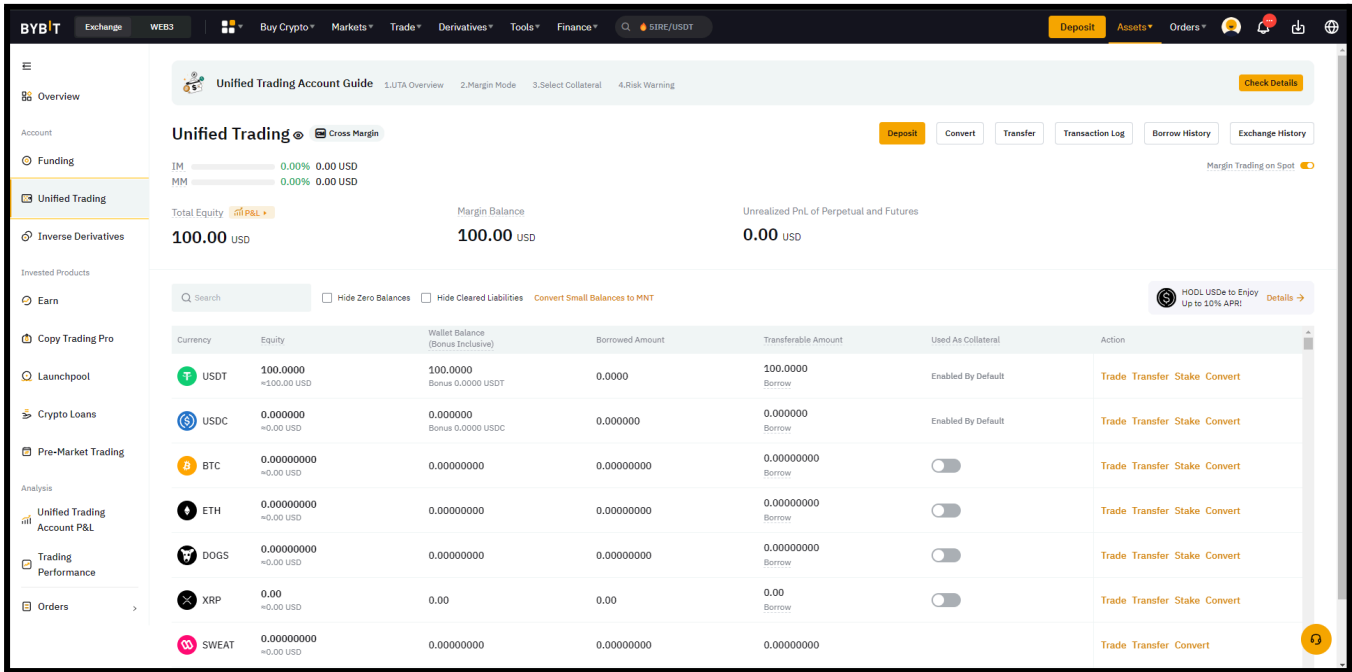
SearchHide Zero Balances

Coin	Net Equity	Wallet Balance (Bonus Inclusive)	Unrealized P&L	Available Balance (Bonus Inclusive)	Position Margin	Order Margin	Bonus	Action
USDT	100.00000 = 100.00 USD	100.00000 = 100.00 USD	0.00000 = 0.00 USD	100.00000	0.00000	0.00000	0.00000	Deposit Loan Convert Transfer Trade
ADA	0.0000000 = 0.00 USD	0.0000000 = 0.00 USD	0.0000000 = 0.00 USD	0.0000000	0.0000000	0.0000000	0.0000000	Deposit Loan Convert Transfer Trade
BTC	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000	0.000000000	0.000000000	0.000000000	Deposit Loan Convert Transfer Trade
DOT	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000	0.000000000	0.000000000	0.000000000	Deposit Loan Convert Transfer Trade
EOS	0.00000 = 0.00 USD	0.00000 = 0.00 USD	0.00000 = 0.00 USD	0.00000	0.00000	0.00000	0.00000	Deposit Loan Convert Transfer Trade
ETH	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000	0.000000000	0.000000000	0.000000000	Deposit Loan Convert Transfer Trade
LTC	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000	0.000000000	0.000000000	0.000000000	Deposit Loan Convert Transfer Trade
MANA	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000	0.000000000	0.000000000	0.000000000	Deposit Loan Convert Transfer Trade
SOL	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000 = 0.00 USD	0.000000000	0.000000000	0.000000000	0.000000000	Deposit Loan Convert Transfer Trade

Appendix 4 : 30 Days PnL of Bybit Account of Bayesian Optimization Implementation



Appendix 6 : Bybit Account of Grid Search CV Implementation



Appendix 6 : 30 Days PnL of Bybit Account of Grid Search CV Implementation

